Metabolic engineering: the study module

Part 2 — The framework: modelling, control and the numbers that decide a design (weeks 11 to 19)

This part is written at a little over twice the density of part 1 — about 2,700 words a week against 1,265 — and deliberately. Phase 1 competes with Lehninger and Alberts, which are better than anything I can compress into five pages a week. Phase 2 competes with nothing you own: Stephanopoulos is excellent and thirty years old, Palsson assumes you want the full constraint-based apparatus, and the parts you most need — the algebra done in your notation, the code that runs, the arithmetic of what a design costs — are scattered across chapters, documentation and papers. So here they are in one place, worked rather than summarised.

What you should expect. Every chapter derives its results rather than quoting them, every number in the figures was computed by a script rather than drawn by hand, and every worked example is one I actually ran. Where a figure rests on assumed parameters rather than measured ones, the caption says so and names the assumption. The code in chapters 12, 13 and 16 is written to be typed in and run, not read; it is short on purpose.

How to use it. Read the chapter before that week's sessions, then do the sessions. The self-checks at the end are the week's exam, and the phase 2 checkpoint in week 19 assumes all of them. One instruction matters more here than in part 1: when a chapter gives a number, reproduce it yourself before you believe me. This is the phase where your chemical-engineering training makes you faster than a biologist, and the way to cash that in is to check the arithmetic rather than to memorise the result.

Ch.	Week	Chapter
11	11	What the field claims to be: Bailey, network rigidity, and the design loop
12	12	Python for modelling I: the linear algebra, the solvers and the fits you will actually use
13	13	Stoichiometric modelling: $S \cdot v = 0$ and what its null space is telling you
14	14	Driving force and transport: whether a step can run, and whether the product can leave
15	15	Metabolic control analysis: why the rate-limiting step is a myth
16	16	Kinetic and dynamic modelling: $dc/dt = S \cdot v(c)$, and what your data can and cannot identify
17	17	Experimental design and statistics: when a difference is real
18	18	Titer, rate, yield: the three numbers and the arithmetic behind them
19	19	Cofactor balancing, redox and toxicity — and the phase 2 checkpoint

What the field claims to be: Bailey, network rigidity, and the design loop

> *Metabolic engineering is not genetic engineering applied to metabolism. The difference is that it measures the network before it changes it, and judges the change by a flux rather than by a band on a gel.*

The definition, and the word that carries it

Bailey's 1991 paper defined the field as the improvement of cellular activities by manipulating enzymatic, transport and regulatory functions with recombinant DNA. Read it carefully and the operative word is *improvement* — a quantity, measured before and after — rather than *modification*. Classical strain improvement also improves cells, by mutagenesis and screening, and it worked: most of the antibiotic and amino-acid titres of the twentieth century came from it. What it could not do was say why. A production strain from a twenty-round mutagenesis programme carries hundreds of mutations, of which a handful matter, and nobody knows which. You cannot transfer that knowledge to the next molecule.

So the claim of the field is epistemic rather than technical: that a strain should be the result of a stated hypothesis about where flux is limited, tested by a measurement that could have falsified it. That is why the discipline's name is engineering and not breeding, and it is also why the field's literature is so preoccupied with flux measurement. If you cannot measure the distribution of flux, you cannot tell a successful design from a lucky mutation.

Two things follow for how you read papers from week 51 onward. First, a paper that reports a titer and a list of genetic changes, with no flux or balance analysis, is a classical strain improvement paper wearing modern clothes — which is not a criticism, but it tells you how much of its reasoning transfers. Second, the useful papers tend to report a failed intervention alongside the successful one, because the failure is where the network told you something.

Network rigidity: the single most useful idea from 1991

Stephanopoulos and Vallino, in the companion paper to Bailey's, asked why overproduction attempts so often fail despite correct biochemistry. Their answer was that metabolic networks resist redistribution of flux at their branch points, and that this resistance is structural rather than accidental. They called it network rigidity, and they classified branch points — nodes — by how much the split ratio moves when you push on it.

A flexible node redistributes freely: push flux toward one branch and it goes. A rigid node holds its split ratio almost regardless of what you do to the enzymes, so the carbon you liberated upstream arrives and then leaves by the route it always used. Three mechanisms produce rigidity, and you can diagnose which one you have.

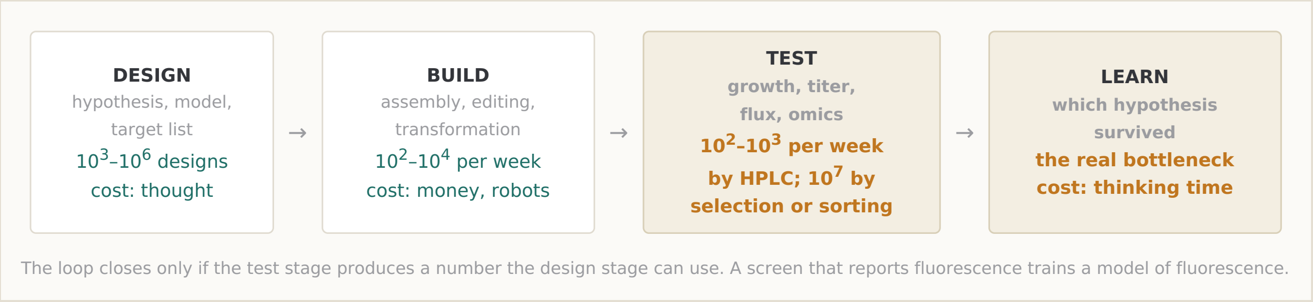
- **Allosteric feedback.** The branch you want is regulated by its own end product, so raising its enzyme raises flux briefly, product accumulates, and the enzyme switches off. You met this in week 3. The fix is a feedback-resistant variant, and it is the single most common intervention in the amino-acid literature.
- **Kinetic competition at the node.** Two enzymes draw on the same metabolite with different K_M values. If the competing enzyme is saturated and yours is not, raising the pool concentration helps yours disproportionately; if yours is the saturated one, raising the pool does nothing at all and you must raise V_{max}. This is decidable from the two K_M values and the measured pool concentration, which is why week 3's arithmetic matters.
- **Thermodynamic constraint.** The branch you want is close to equilibrium, so flux through it is set by the concentration ratio rather than by the enzyme. No amount of expression moves it. Week 14 tells you how to spot this before you build the strain.

The three have different costs to fix, and the diagnosis is cheap compared with the construction. Writing down which of the three you believe you are facing, before you build, is the habit that separates a designed experiment from an expensive guess.

What changed by 2016, and what did not

Nielsen and Keasling's 2016 review is the natural checkpoint against Bailey. What changed is the cost of the build and test steps: DNA synthesis went from a bottleneck to a line item, multiplexed genome editing replaced one-gene-at-a-time construction, genome-scale models exist for every industrial host, and analytics went from one metabolite at a time to a few hundred. What did not change is that a design still fails for reasons the host does not tell you about, that tolerance to the product is still mostly evolved rather than designed, and that the gap between a titer in a flask and a titer in a plant is still measured in years.

The honest summary is that the field got much better at build and test and only modestly better at design and learn. That is worth knowing because it tells you where to point your own effort: the bottleneck is not your ability to make strains.


DESIGN
hypothesis, model, target list
10^3–10^6 designs
cost: thought
→
BUILD
assembly, editing, transformation
10^2–10^4 per week
cost: money, robots
→
TEST
growth, titer, flux, omics
10^2–10^3 per week by HPLC; 10^7 by selection or sorting
→
LEARN
which hypothesis survived
the real bottleneck
cost: thinking time

The loop closes only if the test stage produces a number the design stage can use. A screen that reports fluorescence trains a model of fluorescence.

Figure 11.1 The design–build–test–learn cycle with throughputs attached. The orders of magnitude are indicative of a well-equipped academic lab rather than measured; the point is that they differ by four orders between build and analytical test, which is what biofoundries and selection-based screens exist to reconcile. Drawn for this module.

DBTL as an engineering loop, with arithmetic

Written as a loop, DBTL is familiar: design, build, test, learn, iterate. Treated as an engineering problem it has three parameters — cycle time, parallelism, and information gained per cycle — and the reason to be quantitative about them is that they trade against each other.

A campaign that builds 10^5 variants and screens them by a selection gains enormous parallelism and almost no information: you learn that one variant won, and typically not why. A campaign that builds twelve rationally chosen strains and characterises each with a flux measurement gains little parallelism and a great deal of information, because each strain tests a stated hypothesis. Neither is right in general. What decides is how good your prior is: if your model of the pathway is poor, parallelism buys more than thought, and if it is good, the reverse. Most industrial campaigns run both, with large libraries early and characterised strains late.

Cycle time is the parameter engineers under-rate. A loop that takes six weeks and a loop that takes six days differ by a factor of seven in the number of hypotheses you can test before your funding ends, which compounds more powerfully than any single improvement in hit rate. That is the real argument for automation, and it is the argument you should be able to make in an interview: not that a robot is more accurate than you, but that it shortens the loop.

What counts as evidence in this field

You will read perhaps four hundred papers over this curriculum, and the single skill that compounds is knowing what a claim needs before you believe it. Four claims recur, and each has a standard of evidence you can hold it to.

- **"Enzyme X was limiting."** Needs more than a titer increase on overexpression. The minimum is that the intermediate upstream of X fell and the one downstream rose; better is a titration — several expression levels, showing that the response saturates — because that is the shape control analysis predicts and a single point cannot distinguish from an artefact of the construct.
- **"Flux was redirected."** Needs a flux measurement, not a product measurement. Product up with substrate uptake also up is not redirection, it is more throughput; the carbon balance distinguishes them, and isotope labelling settles it.
- **"The pathway is now balanced."** Needs the cofactor arithmetic shown, per mole of product. A paper that reports a redox fix without stating the demand it was fixing has reported an intervention, not a result.
- **"The strain is stable."** Needs a measurement at the end of the fermentation, not the beginning: plasmid retention without selection, or sequencing of the construct from the final broth, over the number of generations the process actually runs.

Applying these to a paper takes a few minutes and is the main thing the paper log is for. The useful by-product is that it tells you where the field's soft spots are, which is where a new entrant can contribute: the claims that are routinely made without the evidence above are, almost by definition, the ones where a careful measurement is publishable.

A related habit: when a paper reports a fold-improvement, convert it into the two numbers that matter — the absolute titer and the yield as a fraction of theoretical. A tenfold improvement from 0.08 to 0.8 g/L is a result about a research strain; a 20% improvement from 85 to 102 g/L is a result about a process. The literature mixes the two freely and the abstracts rarely distinguish them.

You can build 200 strains a month by hand, or 2,000 a month on the foundry at ten times the cost per strain. Your screen is HPLC at 400 samples a month. Where is the bottleneck, and what should you change?

The bottleneck is the screen, and the foundry would make it worse. Building 2,000 strains a month into a 400-sample screen means five months of backlog accumulating per month of building, so the extra build capacity converts directly into unmeasured strains. Before buying build throughput you buy test throughput, and the cheapest route is almost never a second HPLC: it is a proxy that correlates with the product — a biosensor coupled to fluorescence, a colorimetric assay, a growth coupling — validated against HPLC on a few dozen strains and then used to triage thousands.

The cost of that route is the correlation. A proxy screens what it measures, not what you want, so a library optimised against a biosensor returns variants that optimise the biosensor, including ones that have found a way to light it up without making product. The standard discipline is to confirm every hit by HPLC and to keep the confirmation rate as a reported number, because the day it falls is the day your screen stopped measuring your product.

Three things people get wrong

- ‘DBTL is a project-management diagram.’ It is a loop with a measurable cycle time, and shortening it is the highest-leverage intervention in most campaigns.
- ‘Rigidity means the enzyme is too slow.’ Usually it means feedback, saturation at the node, or a near-equilibrium step — three different fixes.
- ‘Modern tools solved design.’ They solved build. Design and learn are where the field still loses.

Self-check — answer on paper first

1. Give the distinction between metabolic engineering and classical strain improvement in one sentence, then give the consequence for the next molecule you work on.

Metabolic engineering changes a defined target chosen from a measured picture of the network, so the result is attributable; classical improvement selects improved phenotypes without knowing the cause. The consequence is transferability: a mechanism you demonstrated in one strain is a hypothesis you can carry to the next product, whereas a mutagenised strain's advantage is locked inside its genome.

2. You raise the enzyme at a branch point fivefold and the split ratio does not move. Name the three candidate explanations and the measurement that distinguishes them.

Allosteric feedback from the end product, kinetic competition in which the competing enzyme is unsaturated and yours is saturated, and a near-equilibrium step whose flux is set by the concentration ratio. Measure the intracellular concentration of the branch-point metabolite and of the end product: feedback shows as inhibition relieved in a feedback-resistant variant; saturation is decided by comparing the pool concentration with both K_M values; a near-equilibrium step is identified from $\Delta G'$ computed at the measured concentrations, which is week 14's calculation.

3. Your group can either halve the cycle time or double the hit rate of its screen. Which do you argue for?

Halving the cycle time, in most cases, because it doubles the number of cycles and every cycle also raises the hit rate by what you learned in the last one, so the two effects compound. Doubling hit rate is a one-off gain on a fixed number of attempts. The exception is when cycle time is already dominated by the fermentation or the analytics of a necessarily slow test — a sixty-hour fed-batch cannot be shortened by a robot — in which case hit rate is the only handle left.

What to open a book for. Nothing: this week is three papers, and they are short. Read Bailey (Science 252:1668) and Stephanopoulos & Vallino (Science 252:1675) in the same sitting, then Nielsen & Keasling (Cell 164:1185) as the thirty-year report card. If you want the history properly, the introduction chapters of Stephanopoulos, Aristidou & Nielsen cover it from the inside.

Python for modelling I: the linear algebra, the solvers and the fits you will actually use

> *You do not need to learn Python. You need to learn the four or five calls that metabolic modelling actually makes, and to know what each one does when the problem is ill-conditioned, stiff, or under-determined — which is most of the time.*

The environment, and why the versions are part of the result

Create a virtual environment per project, install numpy, scipy, pandas and matplotlib into it, and freeze the versions. This is not hygiene for its own sake: solver defaults change between releases, and a notebook that produced one answer in 2024 can produce a different one in 2026 without any edit. A result you cannot reproduce is not a result, and the cheapest insurance is a `requirements.txt` and a note of the Python version in the README.

One habit worth adopting now, because it will save you in week 51: restart the kernel and run the notebook top to bottom before you believe any figure in it. Notebooks accumulate state, and most wrong figures in science are not wrong calculations but stale variables.

Arrays, and the two mistakes everyone makes

A numpy array is a typed, fixed-shape block of memory; a Python list is not. The two mistakes are shape and dtype. Shape: a vector of length 3 has shape `(3,)`, not `(3,1)`, and broadcasting will silently turn a sloppy combination of the two into a 3×3 matrix rather than raising an error. If a result is suddenly a matrix where you expected a vector, print the shapes, and do it before you print the values. Dtype: an array created from integers stays integer, so dividing it can truncate on some operations and silently promote on others. Write `np.array([...], dtype=float)` for anything numerical.

```
import numpy as np
S = np.array([[ 1, -1,  0,  0],
              [ 0,  1, -1,  0],
              [ 0,  0,  1, -1]], dtype=float)   # 3 metabolites x 4 reactions
v = np.array([1.0, 1.0, 1.0, 1.0])
print(S.shape, v.shape, (S @ v))                # (3, 4) (4,) [0. 0. 0.]
```

The linear algebra metabolic modelling actually uses

Four calls cover almost everything in weeks 13 to 16.

- `np.linalg.solve(A, b)` for a square, determined system. Use this, not `inv(A) @ b`.
- `np.linalg.lstsq(A, b, rcond=None)` for an over-determined system — more measurements than unknown fluxes, which is the good case in flux analysis, because the residual tells you whether your measurements are consistent with your network.
- `np.linalg.matrix_rank(S)` for the number of independent balances, which is what you actually need for a degree-of-freedom count rather than the number of rows.
- `scipy.linalg.null_space(S)` for the space of steady-state flux vectors, which is chapter 13's entire subject.

Why not invert. Inversion computes more than you asked for, costs more, and is less accurate, but the real reason is diagnostic: `solve` will tell you when the matrix is singular, whereas a nearly singular matrix inverts quietly and returns enormous numbers that look like fluxes. Metabolic matrices are frequently nearly singular, because two reactions in your network are nearly the same reaction, or because a metabolite appears in only one balance. Check `np.linalg.cond(A)`: a condition number near 10^k means you can lose about k digits of the 16 a float carries, so 10^{10} is a warning and 10^{14} means your answer is noise.

```
A = np.array([[1.0, 1.0], [1.0, 1.0 + 1e-10]])
print(np.linalg.cond(A))            # ~4e10: ten digits gone
b = np.array([2.0, 2.0])
print(np.linalg.solve(A, b))        # returns something; believe none of it
```

Integrating differential equations

One call: `scipy.integrate.solve_ivp(fun, t_span, y0, ...)`, where `fun(t, y)` returns dy/dt. Note the argument order — t first, then y — which is the opposite of the old `odeint` convention and the single most common source of a model that runs and is wrong. Pass parameters with `args=(p,)` rather than by closing over globals, so the same function can be called by a fitting routine later.

Two features earn their keep. Events stop integration on a condition, which is how you find the time at which a substrate is exhausted rather than inspecting the output afterwards; set `event.terminal = True` and, if you need the crossing direction, `event.direction`. And `dense_output=True` gives you an interpolant you can evaluate anywhere, which is what you want when fitting to data taken at times that are not your solver's steps.

Stiffness is the thing you must recognise. A system is stiff when it contains processes on widely separated timescales, which metabolic models always do: enzyme steps equilibrate in milliseconds and pools fill over minutes. An explicit solver such as RK45 is then forced to take steps set by the fastest process even when nothing interesting is happening, and the cost is brutal. The figure measures it on the standard stiff test problem: RK45 needed 243,050 evaluations of the right-hand side, LSODA needed 305 and BDF 329, for the same tolerances and the same answer. That is a factor of roughly 800.

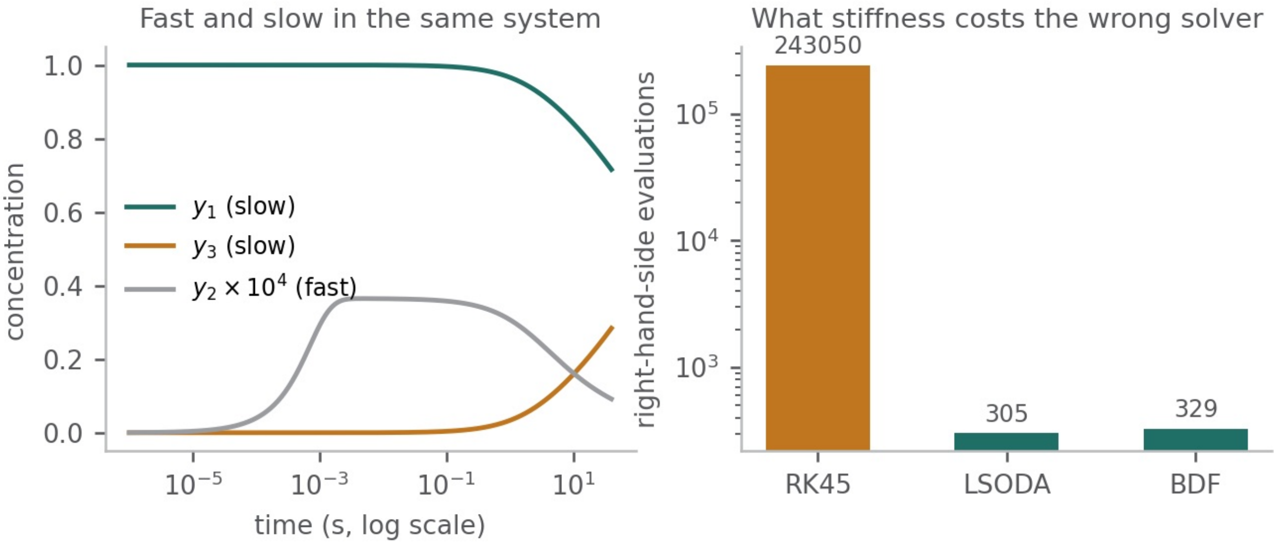


Figure 12.1 Left: the Robertson problem, the standard stiff benchmark, whose middle species moves four orders of magnitude faster than the other two. Right: cost of solving it to the same tolerance (rtol 10^{-6}, atol 10^{-9}) with an explicit and two implicit solvers, counted as right-hand-side evaluations. Computed for this module with SciPy 1.x.

The practical rule: start with `method='LSODA'`, which detects stiffness and switches, and reach for `'BDF'` or `'Radau'` when you know the system is stiff and want the Jacobian used properly. If a model takes minutes to integrate, you have a solver problem and not a biology problem.

Fitting, and reading the uncertainty you get back

`scipy.optimize.curve_fit` returns the parameters and their covariance matrix. The standard errors are the square roots of its diagonal, and the off-diagonal entries matter more than most people notice: normalise the covariance to a correlation matrix and you will frequently find two parameters correlated at 0.99, which means your data constrain their ratio but not either one. This is the first appearance of identifiability, and week 16 treats it properly.

```
from scipy.optimize import curve_fit
def mm(S, Vmax, Km):
    return Vmax * S / (Km + S)

popt, pcov = curve_fit(mm, S_data, v_data, p0=[1.0, 0.5],
                       bounds=([0, 0], [np.inf, np.inf]))
se   = np.sqrt(np.diag(pcov))
corr = pcov / np.outer(se, se)
print(popt, se, corr[0, 1])
```

Three cautions. First, bounds change the problem: with bounds, `curve_fit` switches algorithm, and a parameter pinned at a bound has a meaningless standard error — if V_{max} comes back at exactly its upper bound, the fit failed, it did not succeed. Second, `curve_fit` assumes constant Gaussian error unless you pass `sigma`; enzyme assays usually have error proportional to the rate, so pass `sigma=0.05*v_data` and `absolute_sigma=False` rather than letting the high-rate points dominate. Third, fitting Michaelis–Menten to a transformed linear form — Lineweaver–Burk and its relatives — is a nineteenth-century convenience that distorts the error structure; fit the hyperbola directly.

Five checks that catch most modelling errors

Models fail silently, and the failures are nearly always one of five things. Build the habit of running these before interpreting any output; together they take a minute and they have saved me more time than any debugging skill.

- **Units and scale.** Print the steady-state concentrations and fluxes and ask whether they are physically possible: a metabolite at 10 M or a flux implying a doubling time of 20 seconds means a parameter is in the wrong units. Most model errors are unit errors wearing a biological costume.
- **Conservation.** Sum what should be conserved — total cofactor, total carbon entering against leaving — and confirm it is constant to numerical precision over the integration. A drift means a sign error in S.
- **The zero test.** Set a V_{max} to zero and confirm the flux through that branch goes to zero and that the model does something sensible rather than returning negative concentrations. Negative concentrations are always a modelling error, never a result.
- **The steady-state cross-check.** Integrate to steady state, then solve for the steady state directly with a root finder and confirm they agree. Disagreement means the integration did not converge or there are multiple steady states — both worth knowing.
- **Independence from the initial condition.** Rerun from different starting concentrations. If the steady state moves, you have multistability, which is a real phenomenon but one you should discover deliberately rather than by accident in a figure.

```
from scipy.optimize import fsolve

def steady_state(p, guess):
    f = lambda c: rhs(0.0, c, p)          # dc/dt = 0
    c, info, ok, msg = fsolve(f, guess, full_output=True)
    assert ok == 1, msg
    assert np.all(c > 0), "negative concentration: check the model, not the solver"
    return c, rates(c, p)
```

Note the two assertions. A model that silently returns a failed solve or a negative pool will produce a figure, and a figure is persuasive; an assertion that fires costs you a minute and saves you a conclusion.

A repository someone else can run

The week's deliverable is not code, it is a repository: a README saying what the thing is, a pinned requirements file, one notebook that runs top to bottom from a clean kernel, figures written to files rather than only displayed, and results exported as CSV rather than read off a plot. The test is social, not technical: a colleague clones it and reproduces your figure without asking you a question. That standard is also what week 51 and your capstone will be judged by, and it is much cheaper to adopt now than to retrofit.

Determined system	`np.linalg.solve(A, b) — never inv(A) @ b`
Over-determined	`np.linalg.lstsq(A, b, rcond=None); the residual is a consistency check`
Conditioning	`cond(A) ≈ 10^k means you lose about k of your 16 digits`
Steady-state fluxes	`scipy.linalg.null_space(S); dim = n_reactions − rank(S)`
Integration	`solve_ivp(f, [t0, t1], y0, args=(p,), method='LSODA') # f(t, y)`
Standard errors	`se = sqrt(diag(pcov)); corr = pcov / outer(se, se)`

You fit V_{max} and K_M to eight points, all at substrate concentrations below $0.2 \cdot K_M$. The fit converges and reports $V_{max} = 4.1 \pm 2.8$ and $K_M = 1.9 \pm 1.4$, correlated at 0.997. What do you report?

That the data determine k_{cat}/K_M and nothing else. At substrate far below K_M the rate law collapses to $v \approx (V_{max}/K_M) \cdot [S]$, so the data constrain only the ratio; the fitter can raise both parameters together along that ratio and the residual barely changes, which is exactly what a correlation of 0.997 and relative errors near 70% are telling you. Report V_{max}/K_M with its own confidence interval, state that the individual parameters are not identifiable from this design, and — the important part — say what would fix it: points at and above K_M, which means an experiment, not a better optimiser.

The general lesson, which returns in week 16: a converged fit is not evidence that the parameters are determined. Always look at the correlation matrix, and prefer to report the combination the data actually constrain.

Three things people get wrong

- ‘The fit converged, so the parameters are right.’ Convergence says the optimiser stopped; the covariance says whether the data had anything to say.

- ‘The model is slow because it is big.’ Three ODEs can be slow if stiff and you chose an explicit solver. Try LSODA before you blame the model.
- ‘`inv` and `solve` are the same thing.’ They differ exactly when the matrix is nearly singular, which is when you most need to be told.

Self-check — answer on paper first

1. Your stoichiometric matrix has 12 rows and 15 columns, and `matrix_rank` returns 10. How many degrees of freedom does the system have, and what does the gap between 12 and 10 mean?

Fifteen reactions minus a rank of 10 leaves five degrees of freedom, so five fluxes must be measured or assumed. The gap between 12 rows and rank 10 means two of the metabolite balances are not independent — typically a conserved moiety such as a cofactor pair whose total is fixed, or a metabolite that appears in only one reaction so its balance duplicates another. Counting rows rather than rank would have made you look for two measurements you do not need.

2. Why does passing `args=(p,)` matter rather than reading parameters from the enclosing scope?

Because the same right-hand-side function then has an explicit parameter interface, which is what a fitting routine, a sensitivity analysis and a parameter sweep all need: each must call it with different values without editing the function or mutating a global. The practical symptom of getting this wrong is a sensitivity analysis that reports zero sensitivity, because every call silently used the same global parameter set.

3. Give two signs, visible without any biological knowledge, that a model you have been handed is stiff.

The solver takes a very large number of small steps relative to the simulated interval, visible as a huge `nfev` or a dense `sol.t`; and the solution contains variables whose characteristic times differ by several orders of magnitude, visible as a species that reaches a plateau immediately on a plot where the others are still moving. A third sign is that switching to LSODA or BDF changes the run time by orders of magnitude while leaving the answer unchanged.

What to open a book for. No book. The NumPy absolute-basics and linear-algebra guides, the SciPy integrate and optimize references, and the ten-minute pandas tour are the sources, and they are better than any textbook treatment because they are versioned with the software. Read the `solve_ivp` page in full once; you will use it for the next fifty weeks.

Chapter 13 · tracker week 13

Stoichiometric modelling: $S\cdot v = 0$ and what its null space is telling you

> *$S\cdot v = 0$ is a mass balance around every metabolite in the cell, written once. You have been writing these since your second year; the only new part is that there are two thousand of them and you solve them with a matrix.*

Why the steady state is legitimate

The claim is that intracellular metabolite concentrations are constant, so for every internal metabolite, production equals consumption. That is obviously false in detail — pools move constantly — and it is nonetheless a good approximation for a reason you can quantify. Metabolite pools are small and turnover is fast: a pool at 0.1 mM carrying a flux of 1 mM/s turns over in 0.1 s, while the biomass around it doubles in an hour. Any transient in the pools is over before the culture has changed measurably, so on the timescale on which you make and measure a fermentation, the network is at a pseudo-steady state. The honest statement is that the concentrations are quasi-stationary relative to the growth and feeding timescale, and that is why the approximation is the *pseudo*-steady state rather than the steady state.

It fails where you would expect it to fail: immediately after an induction or a substrate pulse, during a shift between substrates, and for any pool that is not small compared with its flux — a storage polymer, a product accumulating in the cytoplasm. In those cases you need chapter 16.

Building S, and reading it as what it is

Rows are metabolites, columns are reactions, and the entry S_{ij} is the stoichiometric coefficient of metabolite i in reaction j, negative for consumption and positive for production. Then $(S\cdot v)_i$ is the net rate of accumulation of metabolite i, and setting it to zero is the mass balance. Nothing in that sentence is biology.

Take a five-reaction toy network: substrate enters as A, A branches to B and to C, and both branches leave as product.

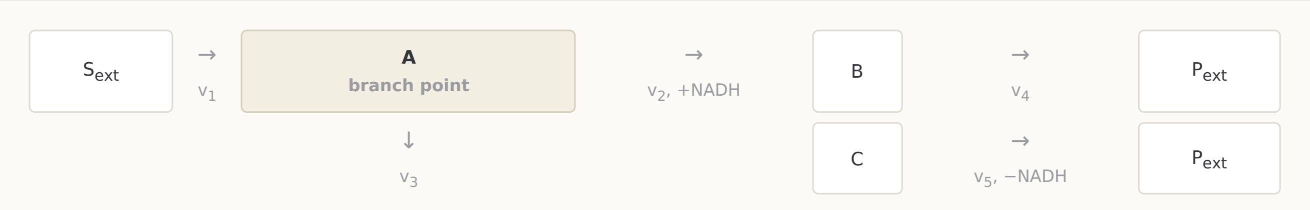
```
  v1: S_ext -> A          A: [ 1 -1 -1  0  0 ]
  v2: A -> B              B: [ 0  1  0 -1  0 ]
  v3: A -> C              C: [ 0  0  1  0 -1 ]
  v4: B -> P_ext
  v5: C -> P_ext          rank(S) = 3, so 5 - 3 = 2 degrees of freedom
```

Solve it by hand: the A balance gives v1 = v2 + v3, the B balance gives v2 = v4, the C balance gives v3 = v5. Two quantities remain free — the overall rate, and how A splits between the two branches. That is exactly what the null space dimension of 2 says, and it is worth pausing on, because it is the general lesson: **the null space is the set of flux distributions consistent with your network, and its dimension is the number of things the network does not determine**. Stoichiometry constrains; it does not decide.

Now add a cofactor and watch the geometry change. Suppose v2 produces one NADH and v5 consumes one, and that NADH must balance like any other metabolite:

```
NADH: [ 0  1  0  0 -1 ]     rank(S) = 4, null space dimension = 1
null space basis, scaled:  v = (2, 1, 1, 1, 1)
```

The split ratio is no longer free. The redox balance has fixed it at one-to-one, because the only NADH sink is the lower branch and the only source is the upper one. This is the thing to take from the week: in real networks the cofactor rows do most of the constraining, which is why a pathway design that ignores them has far more apparent freedom than the cell does. When someone tells you a flux distribution is "possible", ask whether they balanced NAD(P)H and ATP or only carbon.



Without the cofactor: two degrees of freedom — rate and split. With it: one — rate only. Carbon says the split is free; redox says it is not.

Figure 13.1 The toy network of this chapter. Ranks and the null-space basis (2, 1, 1, 1, 1) were computed with `scipy.linalg.null_space` rather than

asserted; reproduce them in six lines before going on. Drawn for this module.

Degrees of freedom, and the three honest ways to close them

Count the degrees of freedom as (number of reactions) − rank(S), not minus the number of rows: rows are frequently dependent, typically through a conserved moiety — the total of NAD^+ and NADH is fixed if no reaction makes or destroys the ring — or because a metabolite appears in only one reaction. Using rank saves you looking for measurements you do not need.

With f degrees of freedom you need f pieces of outside information, and there are exactly three kinds.

- **Measure fluxes.** Exchange rates are the cheap ones: substrate uptake, oxygen uptake, CO_2 evolution, product secretion, growth rate. Each is a boundary flux you can fix from a fermentation balance, and together they frequently close most of the gap. This is the classical metabolic flux analysis route, and it requires nothing but a well-run fermentation.
- **Assume.** Fix a split ratio from literature, set a reaction's flux to zero because the gene is deleted, or constrain a reversible reaction's direction. Assumptions are legitimate and they must be stated, because an assumed flux returns in your answer looking exactly like a measured one.
- **Optimise.** Declare an objective — most often growth — and ask which point in the null space maximises it subject to capacity bounds. This is flux balance analysis, and it is a hypothesis about what the cell is doing, not a deduction from the network. It is remarkably good for wild-type growth on a single substrate and frequently poor for an engineered strain carrying a burden, which is a distinction week 51 will make you defend.

If you have more measurements than degrees of freedom the system is over-determined, which is the good case: solve it with `lstsq` and the residual becomes a consistency check on your measurements. A large residual means your carbon balance does not close, and the usual causes are an unmeasured by-product, a wrong biomass composition, or an uncalibrated dry-weight conversion — all of which you would rather learn now than after you have published the flux map.

Elementary flux modes, and when they are worth the trouble

A null-space basis is mathematically arbitrary: the vectors returned above are orthonormal and contain negative entries, which makes them useless as pathways. Elementary flux modes fix this. An elementary flux mode is a flux vector that satisfies the steady state, respects the irreversibility of every reaction in it, and is minimal, in the sense that removing any of its reactions destroys the steady state. Those three conditions make each mode a genuine route through the network, and every feasible steady-state flux distribution is a non-negative combination of modes.

For the toy network without the cofactor, the modes are the two obvious ones: (1, 1, 0, 1, 0) and (1, 0, 1, 0, 1), the two routes from substrate to product. Add the NADH balance and there is a single mode, (2, 1, 1, 1, 1), which says the only way the network can run at steady state is both branches together in a fixed ratio. Extreme pathways are a closely related construction with extra conventions on how exchange reactions are split; the practical difference rarely matters, and the distinction is worth knowing mainly because both terms appear in the literature.

The catch is combinatorial. The number of elementary modes grows faster than exponentially with network size, and genome-scale networks have more modes than can be enumerated. So modes are a tool for a pathway or a subnetwork of a few dozen reactions — where they are genuinely illuminating, because they tell you every route and therefore every theoretical yield — and not for a whole organism, where constraint-based optimisation is the only tractable approach.

Biomass as a reaction, and the linear programme it makes possible

The step that converts a stoichiometric network into a usable model of a growing cell is the biomass reaction: one artificial reaction that consumes the twelve precursors of week 7, plus amino acids, nucleotides, lipid and cofactors, in the proportions measured for that organism, together with the ATP needed to polymerise them, and produces one gram of dry cell mass. Its flux is the growth rate, in h^{-1}.

Two things about it are worth knowing, because both are routinely misused. First, the coefficients are measurements — of a particular strain, in a particular medium, at a particular growth rate — and they shift with conditions: protein fraction rises with growth rate, which changes the amino acid demand. A model using a 1990s composition for a modern engineered strain in a defined medium is making an assumption you should state. Second, the ATP term splits into growth-associated maintenance, which scales with biomass produced, and non-growth-associated maintenance, a constant drain that the cell pays whether it grows or not. The second is what makes a slow fed-batch expensive: at low growth rate, maintenance consumes a growing fraction of the substrate, and it is why yield on substrate falls as you slow the feed.

With a biomass reaction and capacity bounds, the under-determined network becomes a linear programme:

```
maximise    c·v           (usually the biomass flux)
subject to  S · v = 0     (the balances of this chapter)
            lb ≤ v ≤ ub   (irreversibility, measured uptake, capacity limits)
```

That is flux balance analysis, and recognising it as an LP is most of what you need: the solution sits at a vertex of the feasible region, which is why FBA solutions tend to be sparse and all-or-nothing, and why alternative optima are common — several vertices can give the same objective value with different flux distributions. If you report an FBA flux map, check for alternative optima before interpreting any individual flux, because the one the solver returned is an arbitrary choice among them.

The dual of the LP is where the engineering insight is. Each constraint has a shadow price: the improvement in the objective per unit of relaxation. The shadow price on oxygen uptake tells you what another unit of oxygen transfer is worth in growth or product — directly comparable with the cost of supplying it — and a shadow price of zero tells you a constraint is not binding, so effort spent relaxing it is wasted. This is the same reduced-cost reasoning you would apply to any process optimisation, and it transfers without translation.

Two standard variants are worth naming now because you will meet them constantly. Flux variability analysis fixes the objective at its optimum and then minimises and maximises each flux in turn, returning the range each flux can take without changing the objective — which is the honest way to report an FBA result. And minimisation of metabolic adjustment, and its relatives, replace the growth objective for engineered strains with the assumption that a mutant stays as close as it can to the wild-type flux distribution, which predicts knockout phenotypes better than growth maximisation does, precisely because an engineered cell has not had time to evolve toward optimality.

The balance	`S·v = 0, one row per internal metabolite, one column per reaction`
Degrees of freedom	`f = n_reactions − rank(S); rank, not row count`
Solution space	`v ∈ null(S); dimension f is what the network does not determine`
Over-determined	`more measurements than f → lstsq, and the residual tests your data`
Yield from a mode	`Y_max = max over modes of (product flux / substrate flux)`

Your 42-reaction model of central metabolism has rank 38 and you have measured glucose uptake, oxygen uptake, CO_2 evolution, acetate secretion and growth rate. Is the system determined, and what do you do with the answer?

Forty-two reactions minus rank 38 leaves four degrees of freedom and you have five measurements, so the system is over-determined by one. Solve it as a least-squares problem rather than discarding a measurement: the single redundancy gives you one consistency check, and the residual on it is the only internal evidence you will get that the flux map means anything. If the residual is small, you report fluxes with some confidence; if it is large, one of the five numbers is wrong, and the candidates in order of likelihood are the CO_2 measurement (gas-phase calibration drifts), the biomass composition used to convert growth rate into precursor drains (usually taken from a paper on a different strain), and an unmeasured by-product.

What you must not do is quietly drop the redundancy to make the system exactly determined. That converts your one piece of validation into a number that cannot be wrong, which is the same as having no validation at all.

Three things people get wrong

- ‘The null space gives the flux distribution.’ It gives the set of candidates; measurements or an objective choose among them.
- ‘Degrees of freedom equal reactions minus metabolites.’ Minus the *rank*. Conserved moieties make rows dependent.
- ‘Elementary modes scale to genome models.’ They do not; their number explodes. Use them on pathways.

Self-check — answer on paper first

1. Explain $S \cdot v = 0$ to a chemical engineer who has never seen it, in three sentences.

Each row is a steady-state material balance around one chemical species inside the cell: what is made equals what is consumed, with the stoichiometric coefficients supplying the conversion factors. The vector v is the set of reaction rates, so the matrix product is the accumulation term, set to zero exactly as in any continuous process at steady state. The only unfamiliar part is scale — a few thousand simultaneous balances — and the consequence that the system is usually under-determined, so the balances bound the fluxes rather than fixing them.

2. Your network has one free degree of freedom after all measurements, and the free parameter is the split at a branch point. Give two different ways to resolve it, and state what each one costs in credibility.

Measure it, by feeding a labelled substrate and inferring the split from labelling patterns — expensive in time and instrumentation but it is a measurement. Or assume it, from a published value or by assuming the cell maximises growth, which costs nothing and makes every downstream number conditional on that assumption; the cost is that a reader cannot distinguish your assumed split from your measured fluxes unless you label it clearly, and conclusions that depend on it must be reported with that dependence.

3. Why does adding a cofactor balance usually reduce the apparent freedom of a designed pathway more than adding another carbon reaction?

Because cofactors appear in many reactions across the whole network, so their balance couples parts of the network that share no carbon, whereas a carbon reaction adds one column and one or two entries. A single NADPH balance can tie a biosynthetic branch to the pentose phosphate pathway and to respiration simultaneously, which is why designs that balance only carbon look feasible and designs that balance carbon, redox and ATP usually do not.

What to open a book for. Stephanopoulos Ch. 8, 12 and 13 are the core, and Ch. 12 on determined and under-determined systems is the one to work with a pen. Palsson Ch. 1 to 4 give the same material in the constraint-based language you will meet in the software, and his treatment of the null space is the clearer of the two. Do the algebra by hand first and in numpy second; the agreement is the lesson.

Driving force and transport: whether a step can run, and whether the product can leave

> *Two questions decide more designed pathways than any kinetic argument: can this step carry flux in the direction I need at the concentrations the cell can hold, and can the product get out of the cell at the rate I am making it.*

ΔG′, computed rather than quoted

Week 4 gave you $\Delta G' = \Delta G^{\circ\prime} + RT \cdot \ln Q$. This week you use it on real numbers, and the exercise changes how you read pathways. With classic erythrocyte metabolite concentrations, hexokinase comes out near −27 kJ/mol and phosphofructokinase near −26: both far from equilibrium, effectively irreversible, both regulated. Triosephosphate isomerase comes out near zero, as do phosphoglucose isomerase, aldolase, phosphoglycerate mutase and enolase. Pyruvate kinase is far from equilibrium again, near −14 kJ/mol.

The pattern is the point: the three far-from-equilibrium steps of glycolysis are precisely the three regulated steps. That is not a coincidence, it is a design principle you can use. A reaction near equilibrium cannot be a control point, because its net flux is a small difference between two large opposing fluxes and is set by the concentration ratio rather than by the enzyme; doubling such an enzyme doubles both directions and changes the net almost not at all. A far-from-equilibrium step is where control can sit, and therefore where regulation evolved and where your intervention has a chance.

Two cautions. The values are organism- and condition-specific, so quote your source — and eQuilibrator, which applies a consistent component-contribution estimate and lets you set pH and ionic strength, is the right source rather than a table copied from a textbook. And the classification is about the magnitude of ΔG′, not its sign: a step at −2 kJ/mol runs forward, but only just, and a modest accumulation of its product will stop it.

The driving force of a pathway, and the max-min idea

A designed pathway can have a comfortably negative overall ΔG′ and still be unrunnable, because what stops it is its worst step. The max-min driving force, from Noor and colleagues, formalises this: choose the intermediate concentrations, within physiologically allowed bounds — conventionally 1 µM to 10 mM, with cofactor ratios fixed at measured values — so as to maximise the smallest driving force across all steps. The result is a single number per pathway, and it ranks competing designs.

Read the output rather than the number alone. If the optimum has the smallest −ΔG′ still negative in the wrong direction, the pathway cannot run at any feasible concentration and no enzyme will rescue it. If the optimised minimum is a few kJ/mol, the pathway runs but only with intermediates pinned at the top or bottom of their allowed range, which tells you two useful things: which intermediate must be kept low, and that the design is fragile to anything that lets that intermediate accumulate. And the solution vector tells you which concentrations the design depends on — the diagnostic, not the score, is what you want.

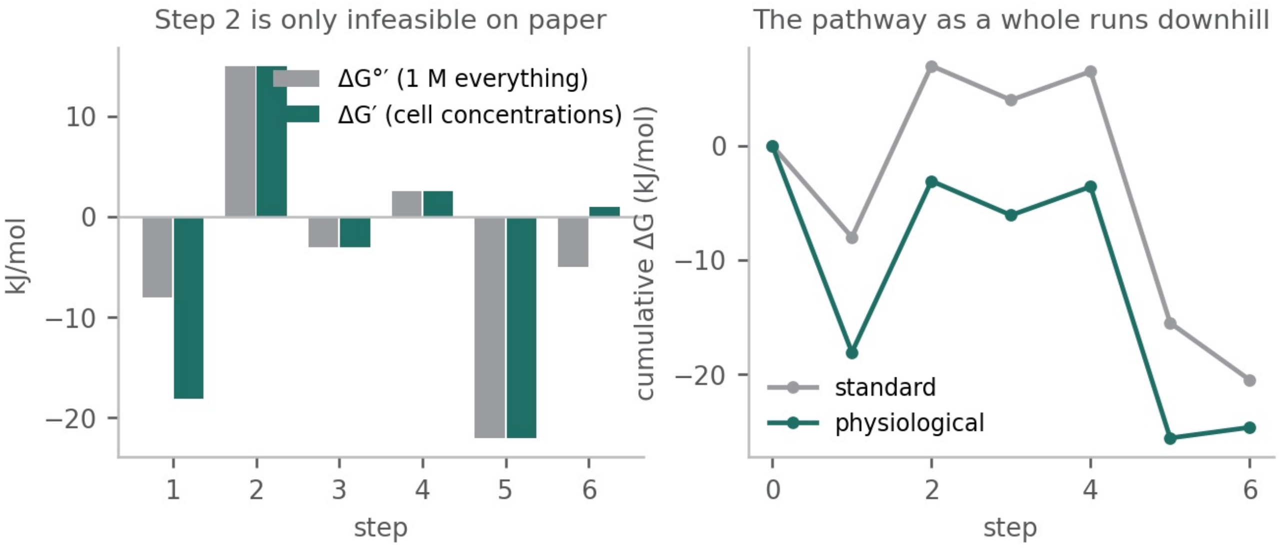


Figure 14.1 A six-step pathway with assumed standard free energies, computed at 37 °C with substrate at 5 mM, intermediates at 0.1 mM and product at 1 mM. Step 2 is positive under standard conditions and negative at cell concentrations — the usual situation and the reason standard values alone never condemn a design. Parameters are illustrative, the arithmetic is not.

Getting out: diffusion, and why it is usually not enough

Fick's law across a membrane is $J = P \cdot \Delta C$, where the permeability coefficient P collects the partition coefficient between water and lipid, the diffusivity in the membrane, and the membrane thickness. The numbers are unforgiving. For a small neutral solute, P is of order 10^{-6} cm/s, which is 10^{-8} m/s; with a 10 mM inside-to-outside difference, that is 10 mol/m³, so $J = 10^{-7}$ mol·m^{-2}·s^{-1}, about 0.36 mmol per square metre per hour.

Whether that is enough depends on how much membrane a cell has per unit of production. A 1 µm cell has roughly 3 µm² of surface, and a cell producing at 2 fmol per hour — about 0.2 g/L/h of a 100 Da product at 10^9 cells per mL — needs a flux of about 1.6×10^{-7} mol·m^{-2}·s^{-1}. So for a small neutral molecule, passive diffusion is of the same order as the requirement: marginal, and it only works because the concentration gradient can be made large. For a polar molecule P falls by two orders of magnitude and for a charged one by four to six, and diffusion stops being an option at all.

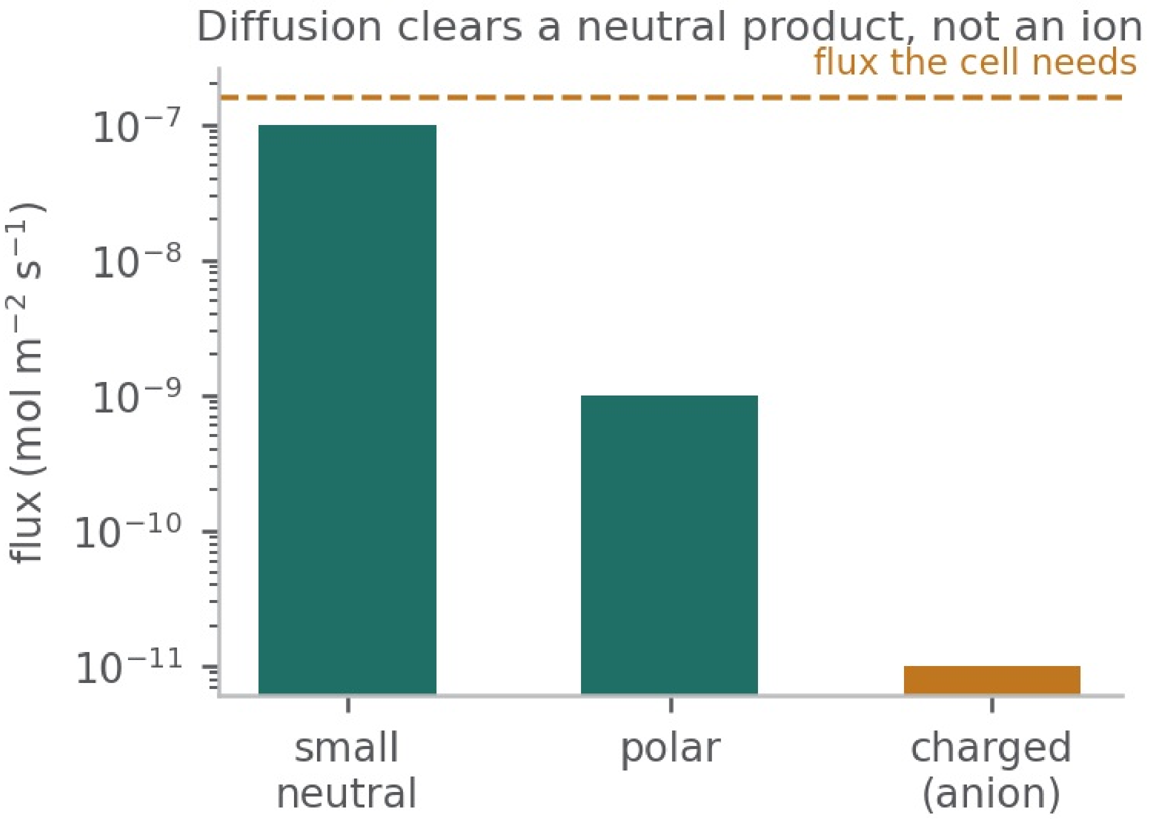


Figure 14.2 Diffusive flux at a 10 mM gradient for three permeability classes, against the flux a producing cell requires. Assumptions, all stated in the text: P = 10^{-8}, 10^{-10} and 10^{-12} m/s; 3 µm² of membrane per cell; 2 fmol per cell per hour. Computed for this module.

This is why the question "is my product charged at cytoplasmic pH?" belongs on the first page of a design. Almost every organic acid, amino acid and phosphorylated compound is, and each therefore needs a transporter, which means either finding one, engineering specificity into one, or accepting intracellular accumulation with the ceiling that part 1 computed.

Weak acids, pH partitioning and the trap

A weak acid crosses as its neutral form and dissociates on the other side, so its equilibrium distribution follows the pH on each side. For an acid of pK_a 4.8 with a cytoplasm at pH 7.6, the ratio of total acid inside to outside is $(1 + 10^{pH_{in} - pK_a})$ / $(1 + 10^{pH_{out} - pK_a})$: about 4 at external pH 7, about 38 at pH 6, and about 245 at pH 5.

Two consequences, which are the same physics read in two directions. For an organic acid product, acidifying the broth — which you may want to do for downstream processing, or which may happen because the product is an acid — traps the product inside the cell as the anion and dissipates the pH gradient, because each neutral molecule that diffuses in releases a proton into the cytoplasm. That is the mechanism of weak-acid toxicity, and it is why acetate at 5 g/L hurts far more at pH 5.5 than at pH 7. For an acid you are feeding or an inhibitor you are trying to exclude, the same arithmetic tells you what the external pH buys you.

Carriers, pumps and what export costs

A transporter is an enzyme with a V_{max}, which is the structural difference from diffusion: diffusive flux rises indefinitely with the gradient, carrier flux saturates. So a strain that works at low titer can hit a transport ceiling at high titer, and the symptom — intracellular accumulation with flux falling as titer rises — looks exactly like product inhibition of the pathway. Distinguishing them requires measuring the intracellular concentration.

The thermodynamic cost of moving product against a gradient is $RT\cdot\ln(c_{out}/c_{in})$: 5.8 kJ/mol per tenfold at 30 °C, so concentrating from 10 mM inside to 1 M outside costs 11.6 kJ/mol. Against ATP hydrolysis at roughly 40 to 60 kJ/mol in vivo, that looks almost free, and in the reversible limit one ATP would move a molecule against many orders of magnitude. Real transporters run far from reversibility, so budget one ATP, or one to two protons of the motive force, per molecule exported, and put it in the yield calculation. For a product made at 1 mol per mol of glucose, one ATP per molecule exported is a few per cent of the cell's energy budget — not decisive, but not negligible either, and it belongs in the balance rather than in a footnote.

One more number: why the cell is well mixed and the vessel is not

The characteristic diffusion time is $t \approx L^2/D$. With $D \approx 5 \times 10^{-10}$ m²/s and L = 1 µm, t ≈ 2 ms, which is faster than most enzyme turnovers: the cytoplasm is well mixed, and treating it as a single compartment in your model is justified. Put L = 10 m into the same expression and you get about 6,300 years. Mixing in a vessel is therefore entirely mechanical, and the gradients in substrate, oxygen and pH that a cell experiences as it circulates are real and are the subject of weeks 40 onward. The same two-line calculation justifies your model's central assumption and condemns the scale-up.

Cofactor ratios are part of Q, and they are not free parameters

Most interesting reactions involve a cofactor pair, and the pair's ratio enters the mass-action ratio directly: for a reduction, Q contains $[NAD(P)^+]/[NAD(P)H]$, so a reaction's driving force depends on the redox state of the pool it draws on, not merely on the metabolites you were thinking about. The two pools are held at deliberately different ratios — the catabolic pool oxidised, the anabolic pool reduced, as part 1 chapter 4 explained — and the difference between their ratios is orders of magnitude.

Three consequences follow, and they are the reason this section exists rather than being a footnote to week 4.

- A reduction driven by NADPH has a substantially larger driving force than the same reduction driven by NADH, simply because the pool it draws on is more reduced. So a cofactor swap made for supply reasons can weaken the thermodynamics of the step you swapped. Check it; it takes two minutes.
- The ratios are set by the cell's own balance of production and consumption, so they are not parameters you can choose in a model. Fix them from measurements for the organism and condition, and when you optimise concentrations for a driving-force calculation, hold the ratios fixed — which is exactly what the standard max-min driving force formulation does, and why.
- Your pathway changes them. A large NADPH demand pulls the anabolic pool toward oxidised, which reduces the driving force of every NADPH-dependent step in the cell, including the ones you did not install. That coupling is invisible in a pathway-level calculation and is one of the standard reasons a design behaves worse in the cell than on paper.

Turning thermodynamics into constraints a model can use

The practical payoff of this week is that $\Delta G'$ becomes a bound rather than an observation. In a constraint-based model, a reaction whose driving force is positive in the forward direction at every feasible concentration can have its lower bound set to zero: you have eliminated a direction, and therefore a whole family of spurious solutions, without any experimental data. Applied systematically this is thermodynamic flux balance analysis, and its main practical effect is to remove the internal cycles that plain FBA permits — loops that carry flux round a set of reactions with no net conversion, which are thermodynamically impossible and which plain FBA will happily return because $S\cdot v = 0$ does not forbid them.

The reverse use is a design filter. Before committing to a heterologous route, compute the max-min driving force for every candidate route to the same product and discard those whose optimised worst step is positive. This costs an afternoon with eQuilibrator and routinely eliminates designs that would otherwise consume a year, which makes it, hour for hour, the highest-return calculation in the curriculum.

Real driving force	$\Delta G' = \Delta G^{\circ\prime} + RT\cdot\ln Q$; RT = 2.58 kJ/mol at 37 °C, 2.52 at 30 °C
Near equilibrium	$\lvert\Delta G'\rvert$ small → net flux set by concentrations, not by enzyme amount
Max-min driving force	maximise $\min(-\Delta G')$ over concentrations in 1 µM–10 mM

Diffusion	$J = P\cdot\Delta C$; $P \approx 10^{-8}$ m/s neutral, 10^{-12} m/s charged
pH partitioning	`c_in/c_out = (1 + 10^(pH_in−pKa)) / (1 + 10^(pH_out−pKa))`
Export cost	$RT\cdot\ln(c_{out}/c_{in})$ = 5.8 kJ/mol per tenfold at 30 °C
Mixing time	$t \approx L^2/D$: 2 ms across a cell, ~6,300 years across 10 m

Your product is a C6 dicarboxylic acid, pK_a values near 4.4 and 5.6, and you are targeting 80 g/L. Walk through the transport questions in the order you would ask them.

First: at cytoplasmic pH 7.6 both carboxyls are deprotonated, so the species inside the cell is a dianion and passive diffusion is excluded — P for a doubly charged species is some six orders below a neutral solute, and figure 14.2's arithmetic says that is five orders short of the required flux. So an exporter is mandatory, not optional, and finding or engineering one is a project in its own right rather than a detail of the pathway.

Second: the cost. Eighty grams per litre of a 146 Da acid is about 0.55 M outside; against an intracellular concentration of, say, 10 mM that is a 55-fold gradient, so RT·ln 55 ≈ 10 kJ/mol at the thermodynamic minimum, and realistically one ATP or one to two protons per molecule. Put that in the ATP balance before you decide the pathway is energetically viable.

Third: pH. The fermentation will be titrated to keep the broth near neutral, which is fortunate, because at pH 5 the mono-protonated form is abundant outside and will diffuse back in, releasing protons and collapsing the pH gradient — the weak-acid toxicity mechanism. This is why organic-acid processes are run either at neutral pH with a base feed and a salt to acidify downstream, or in a host engineered or selected for low-pH tolerance. Both choices are made here, on thermodynamic grounds, not in the plant.

Three things people get wrong

- ‘The pathway is thermodynamically fine, the overall ΔG is very negative.’ What stops a pathway is its worst step, which is what the max-min driving force reports.
- ‘Small molecules diffuse out.’ Neutral ones, marginally. Charged ones, never — and most products are charged.
- ‘Transport is a detail for the process engineers.’ It has a V_{max} and an ATP cost, both of which belong in your yield calculation.

Self-check — answer on paper first

1. Why can a near-equilibrium reaction not be a control point, and what does that imply for target selection?

Because its net flux is the small difference between large forward and reverse fluxes, and that difference is set by how far the mass-action ratio sits from the equilibrium constant. Raising the enzyme scales both directions equally and leaves the net almost unchanged. The implication is that overexpression targets should be chosen from the far-from-equilibrium steps, which is a thermodynamic screen you can run from eQuilibrator before any cloning — and which, combined with the control analysis of week 15, eliminates most candidate targets on paper.

2. A colleague proposes relying on passive diffusion to export a zwitterionic amino acid at 20 g/L. Give the calculation that settles it.

A zwitterion has no appreciable neutral fraction at cytoplasmic pH, so its membrane permeability is in the 10^{-11} to 10^{-12} m/s range. At the most favourable gradient the cell could hold, say 100 mM = 100 mol/m³, $J = P\cdot\Delta C \approx 10^{-10}$ $\text{mol}\cdot\text{m}^{-2}\cdot\text{s}^{-1}$, against a requirement of order 10^{-7} for a producing cell: three orders of magnitude short, and that is before noting that a 100 mM intracellular concentration is itself near the limit of what the cell tolerates. The conclusion is an exporter, and in fact the amino-acid industry's exporters were identified precisely because of this arithmetic.

3. Your intracellular product concentration rises while the pathway flux falls, as titer increases. Name the two explanations and the experiment that separates them.

Either the exporter has saturated — it has a V_{max}, unlike diffusion — or the product is inhibiting an enzyme of the pathway. Both give falling flux with rising internal concentration. Separate them by overexpressing the transporter alone: if flux recovers and the internal pool drops, transport was the ceiling. The complementary test is in vitro, measuring pathway enzyme activity against product concentration to see whether the inhibition exists at the concentrations you measured inside the cell.

What to open a book for. Lehninger Ch. 11 for membranes and transport, read for the mechanisms rather than the numbers, and Ch. 13's thermodynamics section reread with concentrations in hand. The indispensable tool is not a book but eQuilibrator, and the paper to read is Noor et al., PLoS Comput. Biol. 10:e1003483 (2014), which is short and will change how you look at a proposed pathway.

Chapter 15 · tracker week 15

Metabolic control analysis: why the rate-limiting step is a myth

> *Control over a flux is a property of the whole pathway, shared out among its steps, and the shares sum to one. That single sentence, once you believe it with numbers in front of you, rewrites your list of engineering targets.*

Two coefficients, and the difference between them

A **flux control coefficient** measures how much the pathway's steady-state flux responds to a small change in one enzyme:

```
C^J_i = (∂J / ∂E_i) · (E_i / J)     — a fractional response to a fractional change
```

It is dimensionless, it is a property of the system at a particular state, and it is what you want to know before you clone anything: C = 1 means a 10% increase in that enzyme gives 10% more flux, C = 0.1 means it gives 1%, and C = 0 means it gives nothing.

An **elasticity coefficient** measures how much one enzyme's rate responds to a metabolite concentration, with everything else held fixed:

```
ε^i_S = (∂v_i / ∂S) · (S / v_i)     — a local, enzyme-level property
```

The distinction is the whole conceptual content of the subject. Elasticities are local and measurable in a cuvette: they follow from the rate law of the isolated enzyme. Control coefficients are systemic and cannot be measured on an isolated enzyme at all, because they depend on how the rest of the network responds to the pool changes your perturbation causes. Metabolic control analysis is the machinery that computes the systemic quantities from the local ones, and the reason that matters is that an enzymologist's data can be turned into an engineer's answer.

The summation theorem, and the easiest way to believe it

The flux control coefficients of all steps in a pathway sum to one:

```
Σ_i C^J_i = 1
```

The proof is a scaling argument, and you can verify it computationally in a minute. If you multiply every enzyme in the pathway by the same factor λ, every rate scales by λ at fixed concentrations, so the steady state is unchanged and the flux scales by λ exactly. A fractional change of λ in all enzymes therefore produces a fractional change of λ in flux, and since the total response is the sum of the individual responses, the coefficients must sum to one.

I ran exactly that check on the three-enzyme model used for the figure. Doubling all four steps together — three enzymes and the export reaction — changed the flux by 100.0%, as the theorem requires. Perturbing them one at a time by 1% gave control coefficients of 0.327 for enzyme 1, 0.102 for enzyme 2, 0.108 for enzyme 3 and 0.455 for export, summing to 0.993. The 0.7% shortfall is the finite-difference error in a 1% perturbation, not a failure of the theorem, and the fact that it is small is itself a check that the perturbation was small enough to be local.

What the theorem forbids is the thing everybody says. If one enzyme had a coefficient of 1 — a true rate-limiting step — every other enzyme in the pathway would have to have exactly zero, which would mean that halving any of them changed nothing. That is not what happens in real pathways. Control is distributed, usually with a few steps carrying most of it, and the distribution shifts as soon as you perturb it.

The connectivity theorem, and what it is for

The second theorem links the two kinds of coefficient. For a metabolite S, summing over the enzymes that act on it:

```
Σ_i C^J_i · ε^i_S = 0
```

Its practical use is computational: with the elasticities, which you can get from rate laws, the summation and connectivity theorems give you enough simultaneous equations to solve for all the control coefficients without ever perturbing the real cell. That is the classical route, and it is worth working through once by hand on a two- or three-step pathway so that the algebra is not mysterious. In practice, when you have a kinetic model, you will get the same numbers faster by numerical perturbation — chapter 16 does it in five lines — and the theorems then serve as the check that your numerics are right.

There is a qualitative reading of connectivity worth carrying. An enzyme with a large elasticity toward its substrate — one far from saturation, responding steeply to its pool — tends to carry *little* control, because it absorbs pool changes rather than transmitting them. Saturated, insensitive enzymes carry control. This is the opposite of the intuition that the "fastest" enzyme matters least, and it is the reason that measuring a pool concentration against the K_M of the enzyme that consumes it is such a cheap diagnostic.

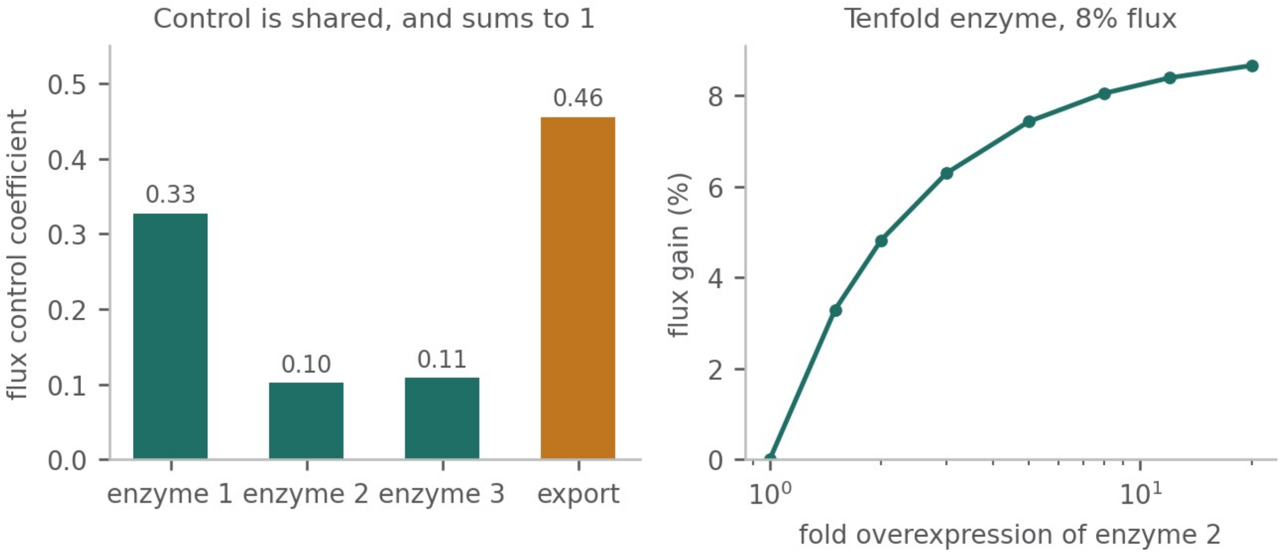


Figure 15.1 Left: flux control coefficients of a three-enzyme pathway with product export and feedback inhibition on step 1, computed by 1% perturbation of each step; they sum to 0.993, with the shortfall being finite-difference error. Right: flux gain against fold overexpression of enzyme 2, which saturates near 8%. Parameters are assumed, the model is in the text of chapter 16, and both panels were computed rather than sketched.

Why overexpressing one enzyme usually fails, in numbers

Enzyme 2 in the model carries a control coefficient of 0.102, so the first 1% of overexpression buys 0.1% more flux. Overexpress it tenfold and you do not get tenfold or even 100% more: you get 8.3%. The curve saturates, and the reason is visible in the model's own output. As $V_{max,2}$ rises, its substrate pool falls from 1.55 to 0.43 in model units while the pool downstream rises, and its own control coefficient collapses from 0.102 to 0.004 — by the time you have overexpressed it twentyfold, that enzyme has no control at all and the limitation has moved somewhere else.

This is the general shape of the result, and it is why the single most common outcome in the literature is a modest gain from a large overexpression. It is also why the answer is not to find the right single enzyme but to change several steps at once — and that is the real argument for multiplexed genome editing in week 23 and for library-based tuning in week 33. Not that editing many genes is impressive, but that distributed control makes it necessary.

One more result from the same model, and it is the most useful one. Removing the feedback inhibition on step 1 — a one-residue change in a real strain — raised flux by 43%, four times what tenfold overexpression of enzyme 2 achieved. And it redistributed control: export control fell from 0.455 to 0.098 while enzyme 3's rose from 0.108 to 0.398. That is what regulation does. It holds control at the demand side, so that the pathway tracks demand rather than supply, and releasing it hands control back to the enzymes — which is exactly why the feedback-resistant variant is so often the first and best intervention, and why the second intervention should be chosen after re-measuring, not from the original analysis.

Where the formalism breaks, and what to do about it

Four limits are worth knowing before you quote a control coefficient at an interview.

- **It is local.** Coefficients are derivatives at one state. A tenfold overexpression is not a small perturbation, and the coefficient that predicted your first 1% does not predict your tenth fold, as the figure shows.
- **Enzyme amount is not V_{max}.** The formalism assumes rate is proportional to enzyme, which fails when the enzyme aggregates, when expression saturates the folding machinery, or when the burden of expression slows growth. In a real strain, overexpression has a cost that the model does not contain.
- **It assumes a steady state exists and is reached.** Where a pathway oscillates or the culture is changing fast, the coefficients are not defined.

- **Boundaries absorb control.** If your model fixes a boundary concentration — substrate held constant, cofactor pool clamped — the control that would have sat on that boundary step disappears, and your coefficients will not sum to one. This is the most common reason a numerical summation check fails, and it is diagnostic rather than fatal.

The two-step case, done by hand

Do this derivation once and the formalism stops being abstract. Take a linear two-step pathway with a fixed external substrate S, one intermediate X, and a fixed external product, with both steps first order in their substrate: $v_1 = k_1(S - X/K_1)$ and $v_2 = k_2X$, where k_i is proportional to enzyme i.

At steady state $v_1 = v_2 = J$, so $k_1S - k_1X/K_1 = k_2X$, giving

```
X = k1·S / (k2 + k1/K1)        and        J = k1·k2·S / (k2 + k1/K1)
```

Differentiate J with respect to k_1 and scale, and after one line of algebra:

```
C^J_1 = k2 / (k2 + k1/K1)          C^J_2 = (k1/K1) / (k2 + k1/K1)
C^J_1 + C^J_2 = 1                  exactly, for any parameter values
```

Three readings, all of which generalise. The summation theorem falls out of the algebra rather than being imposed. The control sits with whichever step is *slow relative to the other* — raise k_2 and control moves to step 1 — which is the migration you saw numerically in the figure. And the equilibrium constant K_1 appears in the answer: the closer step 1 is to equilibrium, the smaller k_1/K_1 has to be before it holds control, which is the thermodynamic argument of week 14 arriving inside the control analysis.

Response coefficients: the version you actually need for a design

Enzymes are not the only thing you change. The response coefficient R^J_P measures how flux responds to an external effector P — an inhibitor, an inducer concentration, a feed rate — and the partitioned response theorem factorises it:

```
R^J_P = C^J_i · ε^i_P     summed over the enzymes that P acts on
```

That is the equation that connects the biochemistry you can measure to the process variable you can set. The effect of an inhibitor on pathway flux is its effect on the enzyme it binds — an elasticity, measurable in vitro — times the control that enzyme holds in the pathway. So a potent inhibitor of an enzyme with little control does almost nothing to flux, which explains a great many disappointing results with pathway inhibitors, and conversely a weak effect on a high-control step can be decisive.

How control coefficients are actually measured

You will see the numbers quoted and should know where they come from, because the method determines how much to trust them.

- **Titratable expression.** Put the enzyme under an inducible promoter, measure flux across a series of induction levels, and take the slope of ln J against ln E at the wild-type point. This is the cleanest method and the most work, and it gives you the whole response curve rather than one derivative — which is better, since the curve is what you need.
- **Specific inhibitor titration.** Titrate an inhibitor of known mechanism and infer the coefficient from the initial slope. Fast, and limited to enzymes with a usable specific inhibitor.
- **Genetic series.** Compare strains with different copy numbers or promoter strengths. The pitfall is that each strain differs in burden as well as in the target enzyme, so some of the response you measure is proteome allocation (week 9) rather than pathway control.
- **From a kinetic model.** Numerical perturbation, as in chapter 16. Free, instant, and only as good as the model's parameters — which is why the answer is a prediction rather than a measurement, and should be described as one.

Flux control coefficient	`C^J_i = (∂J/∂E_i)(E_i/J); dimensionless, systemic`
Elasticity	`ε^i_S = (∂v_i/∂S)(S/v_i); local, from the rate law`
Summation	`Σ_i C^J_i = 1 over all steps, including transport`
Connectivity	`Σ_i C^J_i · ε^i_S = 0 for each metabolite S`

Concentration control	`Σ_i C^S_i = 0: pools are buffered, flux is not`

A reviewer asks why you propose editing six genes when your own model says enzyme 1 carries a control coefficient of 0.33. Answer in a paragraph.

Because 0.33 is the response to an infinitesimal change at the current state, and it decays as soon as the change is made. Raising enzyme 1 alone redistributes control to the steps downstream and to export, so the second 10% buys less than the first; the summation theorem guarantees that no single-enzyme route can reach the flux we need, because the most it can deliver is the fraction of control it holds, and that fraction shrinks as we use it. Six coordinated changes keep each individual perturbation small, which keeps each coefficient near its measured value and keeps the response close to additive. The design is therefore not six guesses but one hypothesis — that control is distributed as measured — tested six ways at once.

The honest addition, which a good reviewer will want: a multiplexed edit also multiplies the ways the strain can go wrong, and the burden of six overexpressions is not in the control analysis at all. That is why the design includes a staged construction with the flux measured after each round, rather than all six at once with a single titer at the end.

Three things people get wrong

- ‘Find the rate-limiting step.’ The summation theorem forbids a single step holding all control in a real pathway.
- ‘Control coefficients are enzyme properties.’ They are system properties, and they change when you change anything, including the thing you measured.
- ‘A coefficient of 0.1 means the enzyme is unimportant.’ It means a small change buys 10% of itself; halving that enzyme may still be catastrophic, because the coefficient is a local derivative, not a global statement.

Self-check — answer on paper first

1. State the summation theorem and its engineering implication, from memory, in the form you would give in an interview.

The flux control coefficients of all the steps in a pathway sum to one. Since control is therefore shared, no single enzyme can deliver more than its share, and using that share redistributes control to other steps, so single-gene overexpression gives sub-linear gains that decay. Useful improvements require several coordinated changes, which is the argument for multiplexed editing and library-based tuning rather than for finding the one correct target.

2. Your numerical control coefficients sum to 1.4. Give three candidate causes and how to test each.

The perturbation was too large, so you measured a secant rather than a derivative: halve it and see whether the sum moves toward one. A conserved moiety or clamped boundary in the model is absorbing or injecting flux: check whether any concentration is held fixed, and whether a cofactor total is conserved, since a conserved pool changes the theorem's form. Or the steady state was not actually reached before you read off the flux: integrate longer, or solve for the steady state directly and compare.

3. Why does an enzyme with a high elasticity toward its substrate tend to carry little control?

Because a steep response to its substrate means the enzyme absorbs changes in that pool: if flux through it rises, its substrate falls slightly and the rate self-corrects, so the perturbation is damped rather than transmitted to the rest of the pathway. Saturated enzymes, whose elasticity is near zero, cannot self-correct, so a change in their amount propagates and they carry the control. This is why comparing a measured pool concentration with the consuming enzyme's K_M is such a cheap way to guess where control sits.

What to open a book for. Stephanopoulos Ch. 11 covers the formalism and has problems worth working all the way through. Fell's *Understanding the Control of Metabolism* is where the theorems are derived properly and is the better book if you want to be able to defend them. Work two or three problems by hand this week; you will do the rest numerically for the next fifty, and the hand calculation is what makes the numerics trustworthy.

Kinetic and dynamic modelling: dc/dt = S·v(c), and what your data can and cannot identify

A kinetic model is the only tool that predicts a transient, and the most expensive one to earn. The judgement this week teaches is knowing when you have earned it and when constraint-based modelling is the honest answer.

From the steady state to the dynamic form

Chapter 13 solved S·v = 0. The dynamic form removes the zero:

```
dc/dt = S · v(c)
```

Everything new is in the parenthesis. In the stoichiometric case v was an unknown vector; here it is a *function* of the concentrations, which means you now need a rate law for every reaction and parameters for every rate law. A 20-reaction model needs perhaps 60 parameters, and that is the whole difficulty of the subject: the structure is free, the parameters are not.

Here is the complete model used for the figures in chapters 15 and 16 — three enzymes, reversible kinetics, feedback inhibition from the product onto step 1, and a first-order export. Type it in and you can reproduce every number I have quoted.

```
import numpy as np
from scipy.integrate import solve_ivp

S0 = 5.0
p = dict(V1=2.0, K1=1.0, K1p=1.0, Keq1=10.0, Ki=0.6, n=2.0,
         V2=2.5, K2=0.5, K2p=0.5, Keq2=5.0,
         V3=2.0, K3=0.5, K3p=0.5, Keq3=8.0, kP=0.6)

def rates(c, p):
    X1, X2, P = c
    v1 = p['V1']*(S0 - X1/p['Keq1'])/(1 + S0/p['K1'] + X1/p['K1p']) \
         / (1 + (P/p['Ki'])**p['n'])            # feedback inhibition
    v2 = p['V2']*(X1 - X2/p['Keq2'])/(1 + X1/p['K2'] + X2/p['K2p'])
    v3 = p['V3']*(X2 - P /p['Keq3'])/(1 + X2/p['K3'] + P /p['K3p'])
    return np.array([v1, v2, v3, p['kP']*P])    # export is the fourth step

def rhs(t, c, p):
    v = rates(c, p)
    return [v[0]-v[1], v[1]-v[2], v[2]-v[3]]

sol = solve_ivp(rhs, [0, 20000], [0.1, 0.1, 0.1], args=(p,),
                method='LSODA', rtol=1e-11, atol=1e-13)
c_ss = sol.y[:, -1]
print(c_ss, rates(c_ss, p))   # X1 1.545  X2 1.338  P 0.787 ; all four rates 0.4722
```

Rate laws, and how much to spend on them

Irreversible Michaelis–Menten is the default and is wrong for most steps in a pathway, because most steps are near enough to equilibrium that the reverse rate matters. The reversible form, $v = V_f(S - P/K_{eq}) / (1 + S/K_S + P/K_P)$, is barely more work and has the property that it cannot violate thermodynamics: flux goes to zero at equilibrium and reverses beyond it. The Haldane relation, $K_{eq} = (V_f K_P) / (V_r K_S)$, ties the kinetic parameters to the thermodynamics you computed in week 14, so fitting them independently is over-fitting: fix K_{eq} from eQuilibrator and let the kinetics respect it.

Beyond that, add only what you can justify: a competitive term for an inhibitor you measured, a Hill exponent for a step you know is cooperative, and nothing else. Convenience kinetics — a generic reversible rate law with one parameter per substrate and product — exists precisely to avoid inventing mechanisms you have no evidence for, and for a model whose purpose is to explore behaviour rather than to predict a number, it is the right default.

Numerics: stiffness, and verifying against the steady state

Metabolic models are stiff, for the structural reason given in chapter 12: enzymatic steps equilibrate orders of magnitude faster than pools fill. Use LSODA or BDF, and treat a slow integration as a solver diagnosis rather than a biological one.

There is one check you should run on every model you build, and almost nobody does: integrate to steady state, then confirm that the resulting flux vector satisfies S·v = 0 to numerical precision and matches what the stoichiometric analysis of chapter 13 predicted. In the model above, all four rates come out equal to 0.4722, as a linear pathway at steady state requires. If they do not, you have a sign error in S or a rate law that does not do what you think, and finding it now is far cheaper than explaining a wrong result later.

Parameter estimation, and the identifiability that bites

The exercise that teaches this is to generate noisy data from your own model and fit the parameters back. You know the truth, so you can see exactly which parameters return and which do not, and the answer is always the same: some come back accurately, some come back with enormous confidence intervals, and some come back wrong with narrow confidence intervals, which is the dangerous case.

Two kinds of non-identifiability, and the distinction matters because the remedies differ. **Structural** non-identifiability is a property of the model: two parameters appear only as a product or a ratio, so no amount of data of that type can separate them. V_{max} and K_M at substrate far below K_M, from chapter 12, is the textbook case; no better experiment of the same design fixes it, only a different measurement. **Practical** non-identifiability is a property of your data: the parameter is identifiable in principle but your measurements are too few, too noisy, or taken at the wrong times. More or better data fixes it.

The diagnostics, in increasing order of effort: look at the correlation matrix from the covariance; re-fit from several starting points and see whether you land in the same place; and compute a profile likelihood for the parameter you care about, by fixing it at a grid of values, re-optimising everything else, and plotting the best achievable residual. A profile that is flat over orders of magnitude is the honest picture of a parameter your data do not determine, and it is the figure to show rather than a confidence interval that implies precision you do not have.

Sensitivity analysis, and getting control coefficients for free

A local sensitivity analysis perturbs each parameter by 1% and records the effect on the output you care about. Do it on the steady-state flux with respect to each V_{max} and you have computed the flux control coefficients numerically:

```
def fcc(p, keys=('V1','V2','V3','kP'), eps=0.01):
    J0 = steady(p)[1][-1]
    out = {}
    for k in keys:
        q = dict(p); q[k] = p[k]*(1+eps)
        out[k] = ((steady(q)[1][-1] - J0)/J0)/eps
    return out            # 0.327, 0.102, 0.108, 0.455 — sum 0.993
```

That is the whole of chapter 15, executed. Check the sum against one, and if it misses, work through the four causes in chapter 15's list before you blame the biology. Global sensitivity analysis — sampling the whole parameter space rather than perturbing around one point — is the honest version when your parameters are uncertain, and ensemble modelling takes it further still: instead of one parameter set, keep a population of sets, all consistent with the available data, and report the spread of predictions. A prediction that is robust across the ensemble is worth acting on; one that depends on which member you picked is not a prediction.

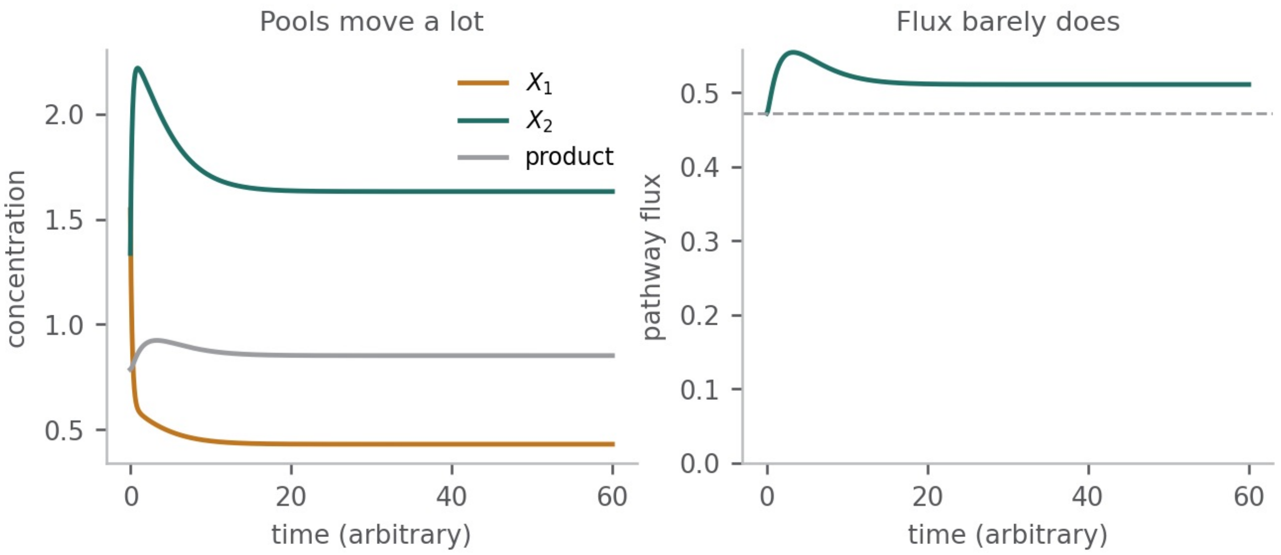


Figure 16.1 The model's response to a tenfold step increase in enzyme 2 at t = 0, starting from the steady state. The intermediate pools move substantially — X_1 falls by a factor of three — while the pathway flux rises by 8%. This is metabolic control analysis seen rather than proved, and it is the

result to expect from a single overexpression. Computed with the model listed above.

Dynamic flux balance analysis, and when to stop modelling kinetically

Dynamic FBA is the pragmatic middle ground: solve a constraint-based model for the current external conditions, use its predicted uptake and secretion rates to advance the extracellular concentrations and biomass by a time step, and repeat. You get substrate depletion, growth, product accumulation and diauxic shifts over a batch, with no intracellular kinetic parameters at all. For simulating a fermentation it is usually the right tool, and COBRApy implements it in a few lines.

So when does a kinetic model earn its parameters? When the question is about a transient, a pool concentration, or a regulatory mechanism — how fast the pathway responds to induction, whether a toxic intermediate accumulates, whether a dynamic controller will be stable. Those questions are invisible to constraint-based methods, which have no concentrations and no time. When the question is about yields, flux distributions at steady state, or the consequences of a deletion on growth, a constraint-based model answers it with assumptions you can state, and a kinetic model answers it with sixty parameters you cannot defend. Choosing the weaker tool is the expert move, and being able to say why in one sentence is a question you will be asked.

Model reduction: making the stiffness go away instead of solving through it

Stiffness comes from timescale separation, and timescale separation is also an opportunity. If a step equilibrates far faster than the rest of the model, you can replace its differential equation with an algebraic one — the quasi-steady-state assumption you already use for reactive intermediates — and the stiffness disappears with it. Michaelis–Menten itself is this reduction applied to the enzyme–substrate complex, which is worth noticing: you have been doing model reduction since week 3 without calling it that.

Two rules make it safe. Reduce only what is genuinely fast at the concentrations your model actually visits, which you check by comparing the reduced model's trajectory with the full one over the perturbation you care about rather than at steady state, where they agree by construction. And remember that the reduction discards the fast dynamics, so any question about the first milliseconds — a rapid-mixing experiment, a burst phase — must be asked of the full model.

The complementary move is non-dimensionalisation: scale concentrations by their K_M values and time by a characteristic turnover time. It costs twenty minutes and pays twice. Numerically, the solver sees variables of order one instead of quantities spanning six decades, which improves conditioning. Conceptually, the parameters collapse into a smaller number of dimensionless groups — ratios of timescales, saturation numbers — and those groups, not the raw parameters, are what the behaviour depends on. A model with fifteen parameters often has four groups that matter, and finding them tells you which measurements are worth making.

Designing the experiment the fit needs

The usual order is backwards: run the experiment, then try to fit it. Reverse it. Given a candidate model, simulate the measurement you are considering and compute the sensitivity of each proposed data point to each parameter — the same 1% perturbation as before, but recorded at each time point rather than at steady state. The result is a map of where in time each parameter is visible, and three things fall out of it.

Points where every sensitivity is near zero carry no information about parameters, however beautiful the data look: typically, long plateaus after the system has settled. Points where two parameters have proportional sensitivity curves cannot separate those two parameters, which is structural non-identifiability diagnosed before the experiment rather than after. And the highest-value sampling times cluster where the sensitivities are largest and least parallel — usually during the transient, which is exactly the part of a time course that is hardest to sample and therefore usually under-sampled.

For a fermentation this analysis typically says: take many samples in the first few hours after the perturbation and fewer later, which is the opposite of the convenient schedule. Knowing that in advance is worth more than any improvement in the fitting algorithm, and it is the sort of argument that gets a chemical engineer taken seriously in a biology group.

Dynamic balance	`dc/dt = S·v(c); same S as week 13, rate laws added`
Reversible MM	`v = V_f(S − P/Keq)/(1 + S/K_S + P/K_P)`
Haldane	`Keq = (V_f·K_P)/(V_r·K_S) — fix Keq, do not fit it`
Numerical FCC	`C^J_i ≈ (ΔJ/J)/(ΔE/E) at 1%; check Σ ≈ 1`
Identifiability	`structural: no data of this kind can help. practical: more data can`

Your model predicts an 8% flux gain from tenfold overexpression of enzyme 2. The experiment gives a 2.3-fold gain. Which is wrong, and how do you find out?

Probably neither the arithmetic nor the experiment: probably the model's structure. A gain that large from one enzyme means its control coefficient in the real cell is far above the 0.1 the model assumed, and the usual structural reasons are that the real pathway has a step the model does not — an intermediate that is unstable or leaks to a side reaction, so that accelerating its consumption recovers carbon the model never lost — or that the model's export term is wrong, since in the model export carries 0.46 of the control and in the cell it may carry much less.

How to find out, in order of cost: check whether the intermediate pool fell as the model predicts, because the pools are where the mechanism shows; measure the product and the by-products, since a gain in product with no change in substrate uptake means carbon was recovered from somewhere and that somewhere is the missing reaction; and re-fit the model with the new data point included, which will tell you whether any parameter set within the existing structure can reproduce both observations. If none can, the structure is wrong, which is the most useful possible outcome — a model that cannot be reconciled with data has told you something, whereas a model you re-fit until it agrees has told you nothing.

Three things people get wrong

- ‘More mechanism is a better model.’ Every term you cannot constrain adds parameters that your data will fit to noise.
- ‘The confidence interval is narrow, so the parameter is right.’ Narrow intervals on correlated parameters are an artefact; look at the correlation matrix or the profile likelihood.
- ‘Kinetic models are the serious ones and FBA is a shortcut.’ They answer different questions; using FBA where it suffices is the stronger answer.

Self-check — answer on paper first

1. What exactly changes between S·v = 0 and dc/dt = S·v(c), and what does it cost you?

Structurally, nothing but the zero: the same stoichiometric matrix, now with accumulation on the left. Conceptually, v becomes a function of the concentrations, so every reaction needs a rate law and every rate law needs parameters — typically three to five each. The cost is that the model's conclusions now depend on numbers you must measure, fit or assume, so a dynamic model's predictions are only as good as its least identifiable parameter, whereas a stoichiometric model's conclusions depend only on the network.

2. Distinguish structural from practical identifiability and give the remedy for each.

Structural non-identifiability means the model's equations determine only a combination of the parameters, such as a ratio, so no quantity of data from that experiment can separate them; the remedy is a different measurement — a different substrate range, a direct measurement of one parameter, or a reparameterisation that fits the combination the data constrain. Practical non-identifiability means the parameter is determinable in principle but your data are too sparse or too noisy; the remedy is more replicates, more time points, or points placed where the model is sensitive to that parameter.

3. A collaborator wants a kinetic model of central metabolism to predict the yield of a deletion strain. What do you advise?

That a constraint-based model answers this question better. Yield at steady state is a stoichiometric quantity, decided by the network and the cofactor balances, and a genome-scale model with the deletion imposed as a bound will give it with assumptions that can be written down. A kinetic model of central metabolism would need hundreds of parameters, most unmeasured for this organism, and its yield prediction would be dominated by those choices. The kinetic model becomes the right tool when the question changes to whether an intermediate accumulates to toxic concentrations, or how fast the strain reaches its new steady state after the deletion.

What to open a book for. Stephanopoulos's chapters on cellular reaction kinetics for the formalism; the COBRApy documentation for dynamic FBA, which is better than any textbook because it is runnable. For identifiability, read a methods paper on profile likelihood rather than a textbook — the subject is younger than the textbooks that mention it. Keep the model listed in this chapter: weeks 31 and 51 both reuse it.

Chapter 17 · tracker week 17

Experimental design and statistics: when a difference is real

Most strain comparisons in the literature are under-powered, and most error bars are drawn on the wrong quantity. This is the cheapest week in the curriculum to get an advantage from, because the arithmetic is easy and almost nobody does it before the experiment.

What an error bar means, and what counts as n

A biological replicate is an independent realisation of the thing you are claiming about: an independent colony, transformed independently, grown in its own flask. A technical replicate is a repeated measurement of the same realisation: the same culture injected three times on the HPLC. They estimate different variances — biological variation plus measurement error, against measurement error alone — and they are not interchangeable.

The consequence is the one people resist. Three flasks inoculated from a single colony are, for the question "is strain B better than strain A", a single biological replicate: whatever that colony happened to be — a plasmid rearrangement, a suppressor mutation, an unusual expression level — is shared by all three flasks and will be reported as if it were the strain. This is pseudoreplication, and it inflates your apparent precision while leaving your actual uncertainty untouched. The practical rule: decide what the independent unit of your claim is, and count that.

Then choose which bar to draw. A standard deviation describes the spread of the population you sampled and answers "how variable are clones of this strain". A standard error, SD/√n, describes the precision of your estimate of the mean and answers "how well do I know this strain's mean". On a figure comparing two strains, the standard error is what the comparison rests on; on a figure characterising variability between clones, the standard deviation is. Say which one the bar is, in the caption, always — a figure whose caption does not say is uninterpretable, and reviewers are entitled to assume the more flattering one was chosen.

Comparing two strains, worked all the way through

Take the tracker's example: strain A at 0.60, 0.64, 0.62 g/g and strain B at 0.65, 0.69, 0.64 g/g, three biological replicates each. The means are 0.620 and 0.660, so the difference is 0.040 g/g — about 6.5% better. The standard deviations are 0.020 and 0.026, the pooled standard deviation is 0.0235, and the standard error of the difference is 0.0191.

The t statistic is 0.040/0.0191 = 2.09 on 4 degrees of freedom, giving p = 0.105; Welch's correction, which you should prefer by default since it costs nothing and does not assume equal variances, gives p = 0.110. The 95% confidence interval on the difference runs from −0.013 to +0.093 g/g. Cohen's d — the difference in pooled standard deviations — is 1.71, which is a large effect by any convention.

Now read those four numbers together, because the reading is the lesson. The effect is large, the p-value is above 0.05, and the confidence interval contains zero but is wildly asymmetric about it: the data are consistent with B being no better, and also with B being 15% better. The honest conclusion is not "no significant difference" — which implies the strains are the same — but "this experiment could not resolve a difference of this size". Those are different claims, and only the second tells a reader what to do next, which is to run more replicates.

What to put in a paper: the effect size and its confidence interval, with the p-value third if at all. A p-value compresses the two things a reader needs — how big the effect is, and how precisely you measured it — into one number from which neither can be recovered.

Power, before the experiment rather than after

Power is the probability that your experiment finds a real effect of a given size. For the comparison above, with d = 1.71, the power at n = 3 is 0.36. That is the sentence worth carrying out of this week: with the standard three replicates, a large, real effect is missed about two times in three. Reaching the conventional 80% requires n = 7, and n = 10 gives 0.95.

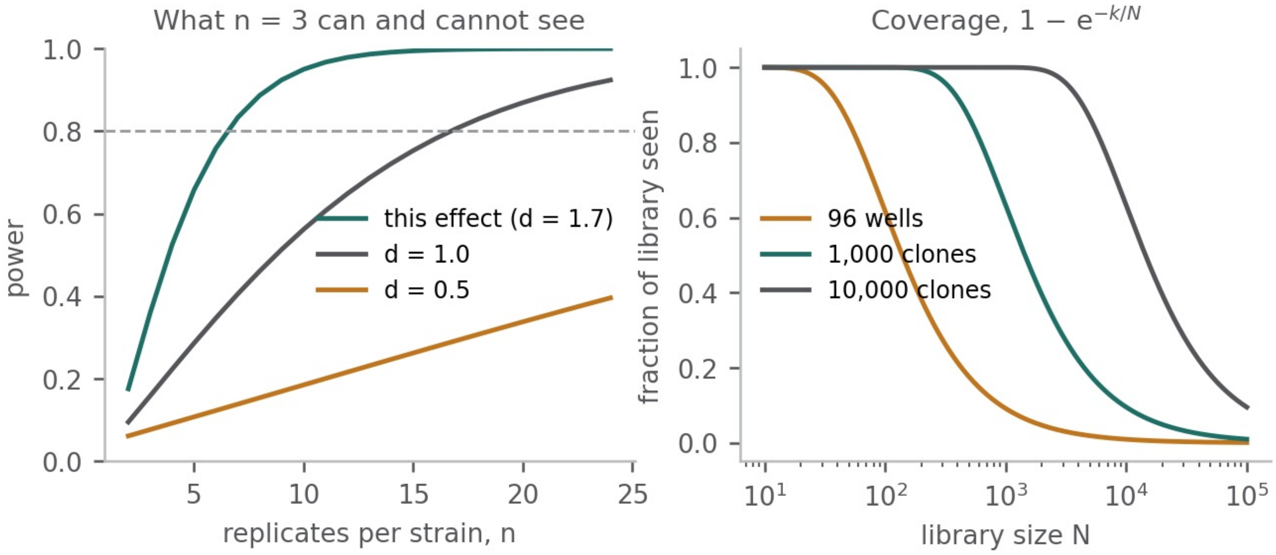


Figure 17.1 Left: power of a two-sample t-test against replicate number for three effect sizes, with the tracker's strain comparison marked (d = 1.71); the dashed line is the conventional 80%. Right: library coverage, $1 - e^{-k/N}$, for three screening efforts. Computed for this module with SciPy; the power curve uses the non-central t distribution rather than a normal approximation.

The constructive use of this is to invert it. Before running the experiment, decide the smallest difference that would change your decision — not the smallest you would like to detect, the smallest that would alter what you do next — and compute the n that detects it with 80% power. If that n is unaffordable, you have learned something important before spending anything: this experiment cannot answer this question, and either the question or the measurement precision has to change.

The same inversion rescues negative results. "We saw no difference" is an absence of information; "we would have detected a 10% difference with 80% power and saw none" is a measurement. Report the second and your negative results become publishable.

Screening many strains: multiplicity and coverage

Two different arithmetics govern screens, and they are often confused.

Multiplicity. Compare 96 strains against a control at $\alpha = 0.05$ and you expect about five false positives even if every strain is identical to the control. If you then pick the best three for follow-up, you have selected the three luckiest wells. The defences are to control the false discovery rate rather than the per-test error, and — more powerful in practice — to re-test hits in an independent experiment, which is the only thing that reliably separates a real effect from a lucky well. Budget the retest from the start.

Coverage. For a library of N variants and k clones screened at random, the expected fraction of the library you have seen at least once is $1 - e^{-k/N}$. A three-gene ribosome binding site library with 20 variants each has N = 8,000, so a 96-well plate covers 1.2% of it, and 1,000 clones cover 12%. Covering 95% needs about 3N — 24,000 clones. This is the calculation that decides whether your screen is a search or a sample, and it is the sentence to have ready in an interview: screening 96 wells of a combinatorial library is not searching the library, it is drawing 96 samples from it, and the right design is therefore either to shrink the library to what you can cover or to couple the phenotype to selection so that the cells do the screening.

Error propagation into a reported yield

A yield is a quotient of two measured quantities, each built from several others, and for independent relative errors the relative errors add in quadrature. List them honestly for a typical yield: HPLC peak area 3%, calibration curve 2%, dry cell weight 5%, culture volume 1%, dilution factor 2%. The total is $\sqrt{(9 + 4 + 25 + 1 + 4)} = 6.6\%$.

Two conclusions. First, a yield of 0.40 g/g carries ±0.026, so it should be reported as 0.40 ± 0.03 and not as 0.4023 — which is the commonest form of overclaiming in the field, and it is visible to any reader who counts significant figures. Second, dry cell weight dominates, contributing 25 of the 43 units of variance, so the way to improve this number is a better biomass measurement, not a better chromatogram. That is a result you can act on, and the calculation that produced it took two minutes.

Design of experiments, in the time it takes to do one factor at a time

Three medium components at two levels each is a 2^3 factorial: eight runs, from which you get all three main effects and all four interactions, each estimated from all eight runs rather than from a pair. The half-fraction, 2^{3-1}, needs four runs and gives you the three main effects — but each is now aliased with a two-factor interaction, so an apparent effect of A is really A + BC and you cannot tell which. That is the trade, stated exactly: fractional designs buy runs with the assumption that high-order interactions are negligible.

One factor at a time is worse than merely inefficient. With interactions present, its answer depends on the order in which you tested, because each factor is optimised at whatever level the others happened to be fixed at; test in a different order and you get a different optimum. And it cannot report an interaction at all, which is a problem when the thing you are optimising — a medium, a feed, an induction profile — is interaction-heavy by nature. The defensible uses are narrow: screening a factor for an effect of any kind at all, or a situation where changing two things at once is physically impossible.

Finally, randomise and block. Plate position, day, batch, operator and instrument are real factors with real effects, and if your strains are laid out so that all of strain A is in the left half of the plate, the edge effect becomes your result. Randomise the assignment, block on the nuisance factors you cannot randomise, and write the plan down before the experiment — a randomisation decided after seeing the data is not randomisation.

The calibration curve, which decides more numbers than the statistics do

Every titer and every yield in your thesis will rest on a calibration curve, and the curve is usually treated as a formality. Four points of practice, in order of how often they are violated.

- **Calibrate over the range your samples occupy, and dilute into it.** Extrapolating above the top standard is the commonest source of a wrong titer, because detector response saturates and the extrapolation is invisible in the fitted line. If a sample reads above the highest standard, dilute and re-inject; do not trust the number.
- **Replicate standards and look at residuals, not R^2.** An R^2 of 0.999 is compatible with obvious curvature, which shows immediately in a residual plot and not at all in the coefficient.
- **Report limits honestly.** With σ the standard deviation of the blank or of the intercept, and m the slope, the conventional detection limit is $3.3\sigma/m$ and the quantification limit $10\sigma/m$. A value between them is evidence that something is there and not a measurement of how much, and should be reported as such rather than as a small number with error bars.
- **Use an internal standard where injection volume or extraction efficiency varies.** It converts a systematic error you cannot see into a ratio you can.

The reason this belongs in a statistics chapter: calibration uncertainty is systematic across every sample in a run, so it does not shrink with replication, and no amount of n fixes it. In the error budget above, the 2% calibration term is the one that stays when everything else has been averaged away.

Multiple comparisons, as a procedure rather than a warning

When you test m strains against a control, the two usable corrections are easy to apply by hand. Bonferroni compares each p-value against α/m: conservative, appropriate when a false positive is expensive — when a hit triggers a fermenter campaign — and nothing else. Benjamini–Hochberg controls the false discovery rate instead: sort the p-values ascending, find the largest i for which $p_{(i)} \le (i/m)\cdot q$, and declare the first i tests significant. With $m = 96$ and $q = 0.10$, the smallest p-value is compared against 0.001, the tenth against 0.0104, and the fiftieth against 0.052.

Which to use is a decision about what the screen is for. In a discovery screen, where every hit will be re-tested anyway, false discovery rate control at $q = 0.1$ to 0.2 is right: you are willing to carry a few false hits into a cheap confirmation step in exchange for not discarding real ones. In a confirmatory experiment, where a positive result is the conclusion, Bonferroni or a pre-registered single comparison is right. Deciding this before you look at the data is what makes either defensible.

Standard error	`SE = SD/√n; SE on comparisons, SD on variability`
Difference	`t = Δ/SE_Δ; report Δ and its 95% CI, not p alone`
Effect size	`d = Δ / pooled SD; d = 1.71 here, and still p > 0.05 at n = 3`
Power	`n = 3 gives 0.36 at d = 1.71; 80% needs n = 7`
Coverage	`fraction seen = 1 − e^(−k/N); 95% needs k ≈ 3N`
Propagation	`(σ_Y/Y)² = Σ (σ_i/x_i)² for independent relative errors`
Aliasing	`2^(3−1): A is confounded with BC, B with AC, C with AB`

A colleague reports that their new strain is significantly better, p = 0.03, from three flasks per strain, and wants to take it to the 5 L fermenter. What do you ask?

First, what the three flasks were: if they came from one colony each, the comparison is sound as far as it goes; if all three came from one colony per strain, n is 1 and the p-value is meaningless regardless of its size. Second, the effect size and its confidence interval, because at n = 3 a p-value of 0.03 implies a very large observed difference and a correspondingly wide interval — the lower bound is what the fermenter campaign will actually deliver. Third, whether this was the only comparison made: if it was the best of twelve strains tested, the effective α is twelve times larger and p = 0.03 is unremarkable.

None of this is an argument against the fermenter run, which may be worth doing on a weak signal if the run is cheap relative to the information. It is an argument for stating the expectation honestly before the run, so that a result at the low end of the interval is recognised as consistent with the flask data rather than as a scale-up failure.

Three things people get wrong

- ‘n = 3 is standard, so n = 3 is enough.’ At a large effect size it has 36% power. Standard is not the same as adequate.
- ‘Not significant means no difference.’ It means this experiment could not resolve one; the confidence interval says what sizes remain possible.
- ‘Error bars are error bars.’ SD and SE differ by √n, and a caption that omits which is being shown cannot be read.

Self-check — answer on paper first

1. Why are three flasks from one colony n = 1, and what is the practical remedy when colonies are expensive?

Because the claim is about the strain, and all three flasks share whatever is peculiar to that one colony — a plasmid rearrangement, a second-site mutation, an unusual expression state — so they measure flask-to-flask variation, not strain variation. The remedy when colonies are expensive is a nested design: a few independent colonies, each with a few flasks, analysed so that the colony-level variance is the one used for the comparison. That recovers the correct error term without demanding many independent colonies, and it also tells you how much of your variance is clonal, which is useful in itself.

2. You screen a 10,000-member library in two 96-well plates and find a variant three times better than the parent. What can and cannot you conclude?

That you have found a better variant, which is a real and useful result. You cannot conclude anything about the library: 192 clones cover under 2% of 10,000, so the best variant in the library is almost certainly not in your sample, and the distribution you observed is a sample of 192 from a distribution you have barely touched. You also cannot conclude the variant is three times better until it is re-tested independently, since a single well at the tail of a 192-well distribution is exactly where measurement noise concentrates.

3. Your yield has 6.6% relative uncertainty, dominated by dry cell weight. Give two ways to reduce it and say which is cheaper.

Replicate the dry-weight measurement, which reduces its contribution by √n and needs nothing but more filters and time; or replace it with a better-calibrated proxy, such as an optical-density-to-dry-weight calibration built from many paired measurements, which shifts the uncertainty from a single noisy measurement into a calibration curve that can be made very precise once and reused. The second is cheaper over a campaign and is the standard practice; the first is the right answer for the one definitive measurement you report at the end.

What to open a book for. No textbook; the *Points of Significance* columns in Nature Methods are the best short treatment and each takes ten minutes. Use scipy.stats for the tests and statsmodels when you need power calculations or a designed experiment analysed properly. If you want one book, a design-of-experiments text aimed at process engineers will serve you better than a biostatistics one.

Titer, rate, yield: the three numbers and the arithmetic behind them

> *Every strain is eventually reduced to three numbers, and which of them binds is decided by the product's price, not by the biology. Knowing which one you are optimising is the difference between a project and a hobby.*

The definitions, with units, and what a flask actually measures

Titer is the product concentration in the broth at harvest, in g/L. **Rate** is volumetric productivity, g/L/h, usually quoted as titer divided by total fermentation time — including the lag and any growth phase, which is the honest version. **Yield** is product per substrate consumed, in g/g, mol/mol, or as a fraction of the theoretical maximum, which is the most informative form because it says how much room is left.

Specific productivity, q_P, in g per g of biomass per hour, is the fourth number and belongs with them: volumetric productivity is $q_P \cdot X$, so a strain can improve on one and worsen on the other, which is the trade you met in week 9.

A shake flask measures titer and yield reasonably and rate badly. It has no pH control, no dissolved-oxygen control and no feed, so the culture spends part of its time in conditions a fermenter would never allow, and the time axis is therefore not comparable to a controlled process. Treat flask rates as a screening ranking rather than as a number, and never extrapolate them to a plant.

Theoretical maximum yield, computed twice

The theoretical maximum is a stoichiometric statement: the best any strain could do with this pathway, with all carbon not required for the product's own chemistry conserved, and with carbon, redox and ATP balances closed. Compute it before you build, because it tells you what fraction of the achievable you have reached and therefore whether to keep working on the strain or on the process.

Ethanol. One glucose gives two pyruvate, each decarboxylated to acetaldehyde and reduced to ethanol, so 2 mol per mol: 2 × 46.07 / 180.16 = 0.511 g/g, with two CO_2 carrying the lost carbon. The redox balance closes exactly — glycolysis makes 2 NADH, the two reductions consume 2 NADH — which is precisely why this is a fermentation and can run anaerobically with no electron acceptor. Real processes reach roughly 90 to 93% of the maximum, the gap going to biomass and glycerol, which is itself the cell's redox valve.

Isobutanol, as a case where the balance does not close so comfortably. Two pyruvate condense to acetolactate, losing one CO_2; reduction and dehydration give 2-ketoisovalerate; decarboxylation loses a second CO_2 and a final reduction gives isobutanol, $C_4H_{10}O$. So one mole per mole of glucose: 74.12 / 180.16 = 0.411 g/g. Carbon closes — six carbons in, four out plus two CO_2. Redox closes in total: glycolysis supplies 2 NADH and the pathway consumes two hydrides. But one of those reductions is NADPH-dependent in the natural enzyme, so the cell must convert one NADH to NADPH per product molecule, and the real cost of that conversion is chapter 19's subject. The arithmetic says the yield is reachable; the cofactor identity says what it will cost to reach it.

The general method, in order, and it is the one the phase 2 checkpoint asks for: write the balanced synthesis from glucose, close carbon, then close NADH and NADPH *separately*, then close ATP. A surplus of NADH means the pathway is more oxidised than the substrate and needs an electron sink or respiration; a deficit of NADPH means pentose phosphate flux or a transhydrogenase, at a cost you must state. Doing redox as one lumped "reducing equivalents" line is the single most common error in a back-of-envelope balance, and it hides exactly the problem that will dominate the project.

Where the money is: two plots you can do on an envelope

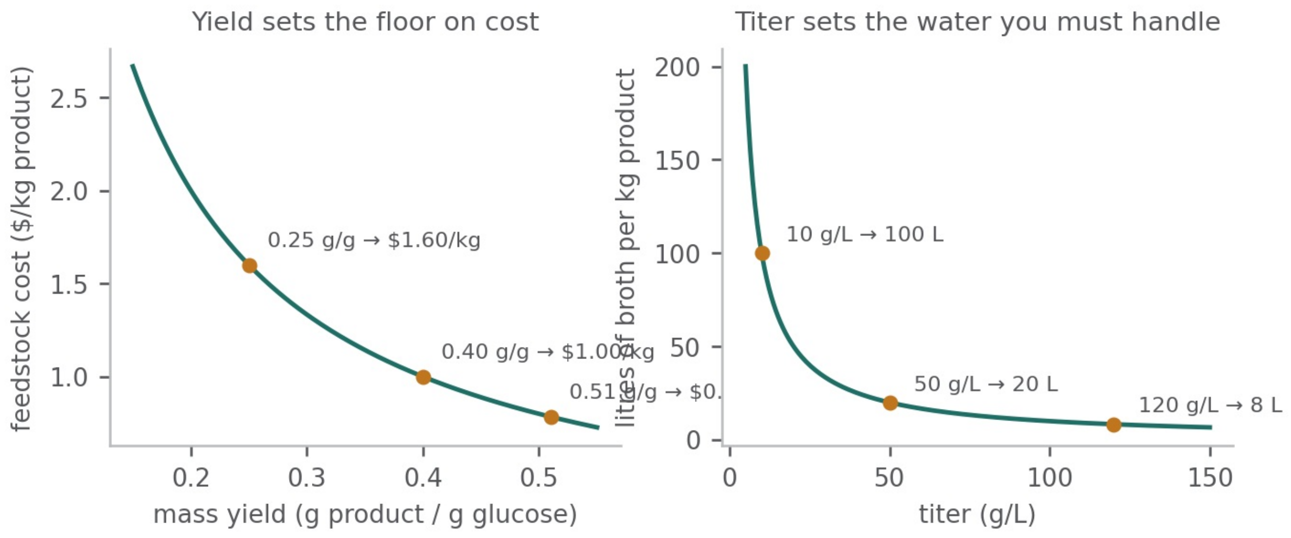


Figure 18.1 Left: feedstock cost per kilogram of product against mass yield, at an assumed glucose price of $0.40/kg; the three marked points are a mediocre strain, a good one and ethanol's theoretical maximum. Right: litres of broth that must be processed per kilogram of product against titer. Both are arithmetic, not data; the glucose price is the only assumption and it moves the left panel vertically without changing its shape.

Feedstock cost per kilogram of product is simply the substrate price divided by the mass yield. At $0.40/kg glucose, a yield of 0.25 g/g means $1.60 of sugar per kilogram of product, and 0.40 g/g means $1.00. For a commodity selling at $1.50 to $2.00/kg, the difference between those two strains is the difference between a business and a demonstration — which is why yield is the binding metric for bulk chemicals, and why the first question about any commodity paper is the yield as a fraction of theoretical, not the titer.

Titer binds differently: it sets the volume of water you must handle per unit of product. At 10 g/L you process 100 litres of broth per kilogram; at 120 g/L you process 8.3. Every one of those litres must be heated, separated, and disposed of, and downstream processing is typically the larger half of the capital cost for a dilute product. This is the quantitative version of a sentence you will read everywhere: a low titer does not merely cost more, it changes which separation technology is feasible at all.

For a high-value pharmaceutical, neither dominates. A few per cent of sugar is irrelevant against a product worth thousands per kilogram, and what matters is total output, purity, and the regulatory reproducibility of the process. The engineering conclusions invert accordingly: for a commodity you optimise yield and accept complexity in the strain; for a pharmaceutical you keep the strain and the process simple and conservative, because a change costs a regulatory filing.

The gap between a paper and a plant

The 1,4-butanediol work of Yim and colleagues is the standard teaching case because the route was designed computationally, built, and then taken to commercial scale, so the whole gap is documented. The paper's titer is tens of grams per litre; commercial operation required several times that, plus years of strain and process development that the paper does not contain. Nothing went wrong in that story — that is what the gap always looks like.

Where does the gap come from? Oxygen transfer falls as vessels get larger; mixing times go from seconds to tens of seconds, so cells experience gradients in substrate, oxygen and pH; the number of generations between the seed vial and the production vessel roughly doubles, which is week 10's genetic instability problem; and the feed profile that was hand-tuned in a 1 L fermenter has to be re-tuned for a vessel whose heat and oxygen limits are different. None of these is a biology problem you can fix in a flask, which is why the industrial literature reads as though it were about a different subject from the academic one.

By-products, and how they destroy a yield while the titer looks fine

The usual suspects in an *E. coli* fermentation, and where each comes from: acetate, from overflow when glycolytic flux exceeds respiratory capacity (week 6); lactate and formate, from fermentative routes under oxygen limitation, including the micro-anaerobic pockets of a large vessel; succinate, from the reductive branch of the TCA cycle under the same conditions; pyruvate, when its consumption is slower than its supply, which is common after a pathway deletion; and glycerol or other reduced compounds, as redox valves when NADH accumulates.

The reason to know the list is diagnostic. Each by-product is an arrow on the metabolic map, so measuring which one appears localises the imbalance: acetate says the feed outran respiration, lactate says oxygen, pyruvate says the step you deleted had a job, glycerol says redox. A fermentation in which you measure only glucose and product is a fermentation in which you cannot diagnose anything, and the marginal cost of adding organic acids to the HPLC method is an hour.

What rate buys: the arithmetic nobody writes down

Yield sets the cost of sugar and titer sets the cost of water; rate sets the cost of steel. Work it through once and you will never again treat volumetric productivity as the third number.

Take a 100 m³ fermenter — 100,000 L — running at 1 g/L/h. That is 100 kg of product per hour, so with 8,000 operating hours a year and 80% of them productive after cleaning, sterilisation, filling and turnaround, the vessel makes about 640 tonnes a year. Halve the productivity to 0.5 g/L/h and the same vessel makes 320 tonnes, so meeting the original target needs a second fermenter, a second seed train and the air, cooling and downstream capacity to match. Rate does not appear in the cash cost per kilogram directly; it appears in how much plant you had to build to produce the kilograms at all, which lands in the capital charge and is usually the largest single line after feedstock.

This is also where the three metrics stop being independent. Pushing titer by extending the fermentation lowers the rate, because the denominator is total elapsed time: a run reaching 100 g/L in 60 h and one reaching 130 g/L in 100 h have productivities of 1.7 and 1.3 g/L/h, and which is better depends on whether the plant is capital-limited or separation-limited. Pushing yield by slowing the feed to suppress overflow lowers the rate too, for the reasons in week 6. There is no strain that is best on all three, and the useful question in any project is which one your particular process is currently losing money on.

A serviceable cash-cost model you can build in a spreadsheet in an hour, and which is enough to keep a conversation with a process economist honest:

```
cost per kg = substrate price / yield            (feedstock)
            + other variable cost per kg          (nutrients, base, enzymes, utilities)
            + annual fixed cost / annual output   (labour, maintenance, capital charge)

annual output = volume x titer x utilisation / batch time, summed over vessels
```

Put your own numbers in and note which term dominates. For a commodity it is nearly always feedstock, which is why yield binds; for a mid-value specialty it is often the fixed term divided by a small output, which is why rate binds; for a biologic it is almost always downstream and quality, which is why neither of the first two appears in the discussion.

Titer	`g/L at harvest`
Rate	`volumetric productivity g/L/h = q_P · X; quote total elapsed time`
Yield	`g/g, mol/mol, and as a fraction of theoretical — the informative form`
Ethanol maximum	`2 mol/mol = 0.511 g/g, redox closed exactly`
Isobutanol maximum	`1 mol/mol = 0.411 g/g, redox closed but cofactor-mismatched`
Feedstock cost	`substrate price ÷ mass yield: $0.40/kg ÷ 0.40 g/g = $1.00/kg product`
Broth volume	`1,000/titer litres per kg: 100 L at 10 g/L, 8.3 L at 120 g/L`

Your strain reaches 12 g/L at 0.18 g/g in 72 h. A colleague proposes a campaign to double the titer. The product is a commodity selling near $1.80/kg. What do you propose instead, and with what numbers?

Yield, and the arithmetic settles it. At 0.18 g/g the feedstock alone costs $0.40/0.18 = $2.22 per kilogram of product against a selling price of $1.80: the process loses money on sugar before any capital, labour, utilities or separation. Doubling the titer to 24 g/L at the same yield halves the water to process, which is worth real money downstream, and leaves the feedstock cost unchanged, so the process still loses money. Raising the yield to 0.40 g/g brings the sugar cost to $1.00/kg and makes everything else worth arguing about.

The constructive version of the proposal: find out where the carbon currently goes, with a closed carbon balance including by-products and biomass, because at 0.18 g/g — probably a quarter of theoretical — the large majority of the carbon is going somewhere identifiable. Then yield work has a target rather than a slogan. Titer becomes the priority once yield is within reach of theoretical, and at that point the usual levers are process rather than genetic: fed-batch profile, in situ removal, tolerance.

Three things people get wrong

- ‘Titer is the headline number.’ For a commodity, yield decides viability and titer decides separation cost; titer alone tells you almost nothing.
- ‘Volumetric productivity is titer over fermentation time.’ Only if that time includes the lag and growth phase. Quoting

the production phase alone flatters the number by a factor of two.

- ‘A clean broth means good yield.’ Carbon can disappear into biomass and CO_2 without a single by-product peak. Close the carbon balance.

Self-check — answer on paper first

1. Compute the theoretical maximum mass yield of ethanol on glucose and state the balance that makes it a fermentation.

Two moles of ethanol per mole of glucose, so 2 × 46.07/180.16 = 0.511 g/g, with the remaining two carbons leaving as CO_2. It is a fermentation because the redox balance closes internally: glycolysis generates 2 NADH, and the two reductions of acetaldehyde consume 2 NADH, so no external electron acceptor is needed and the route can run anaerobically. Any product whose balance does not close this way requires respiration or an electron sink, which is why the list of true fermentation products is short.

2. Explain, with a number, why a by-product can destroy a yield while the titer looks acceptable.

Because titer measures what you made and yield measures what it cost. A strain making 30 g/L of product while excreting 20 g/L of acetate has consumed sugar for both; if the acetate represents a third of the carbon taken up, the yield is a third below what the same titer would imply in a clean fermentation. The titer is unchanged by the leak, and for a commodity the leak is the whole economics. It is also recoverable: the carbon going to acetate is carbon you could redirect, which is why the by-product measurement is the first place to look for yield.

3. Which metric would you optimise for a monoclonal antibody, and why is the answer different from a bulk chemical?

Total output and product quality — the correct glycosylation, aggregate content, reproducibility — with titer mattering mainly because it sets the scale of the purification train. Yield on glucose is nearly irrelevant: sugar is a negligible fraction of the cost of a product selling for orders of magnitude more per kilogram. The inversion is driven entirely by price, and it is why strategies that are obviously right for ethanol are obviously wrong for a biologic.

What to open a book for. Doran's material- and energy-balance chapters, for the accounting done in engineering notation with problems you can work. Read Yim et al., Nat. Chem. Biol. 7:445 (2011) for the numbers rather than the pathway, and log it in the paper log — it is the case study you will be asked about.

Cofactor balancing, redox and toxicity

> *This is the week where phase 1's biology and phase 2's arithmetic meet. Almost every design that fails for a reason nobody predicted fails here: the carbon balanced, the redox did not, and the intermediate was toxic.*

Diagnosing the imbalance, in a fixed order

Take your target's synthesis from glucose and work through four balances in this order, because each one uses the result of the last: carbon, NADH, NADPH, ATP. Keep the two hydride carriers separate — this is the whole point — and express the result per mole of product, so it composes with the yield calculation of week 18.

Three outcomes, each with a standard reading. A **surplus of NADH** means your pathway is more oxidised than the substrate, so the cell must dispose of electrons: aerobically through respiration, which is usually fine and costs oxygen transfer; anaerobically it needs an electron sink, which means a by-product and a yield loss. A **deficit of NADPH** is the common case for reduced products, and it is paid for in carbon or in energy, as below. A **deficit of ATP** means there is nothing left for growth and maintenance, which is a red flag rather than an impossibility — it usually means the process must be two-stage, with growth paid for separately from production.

The three fixes, and what each actually costs

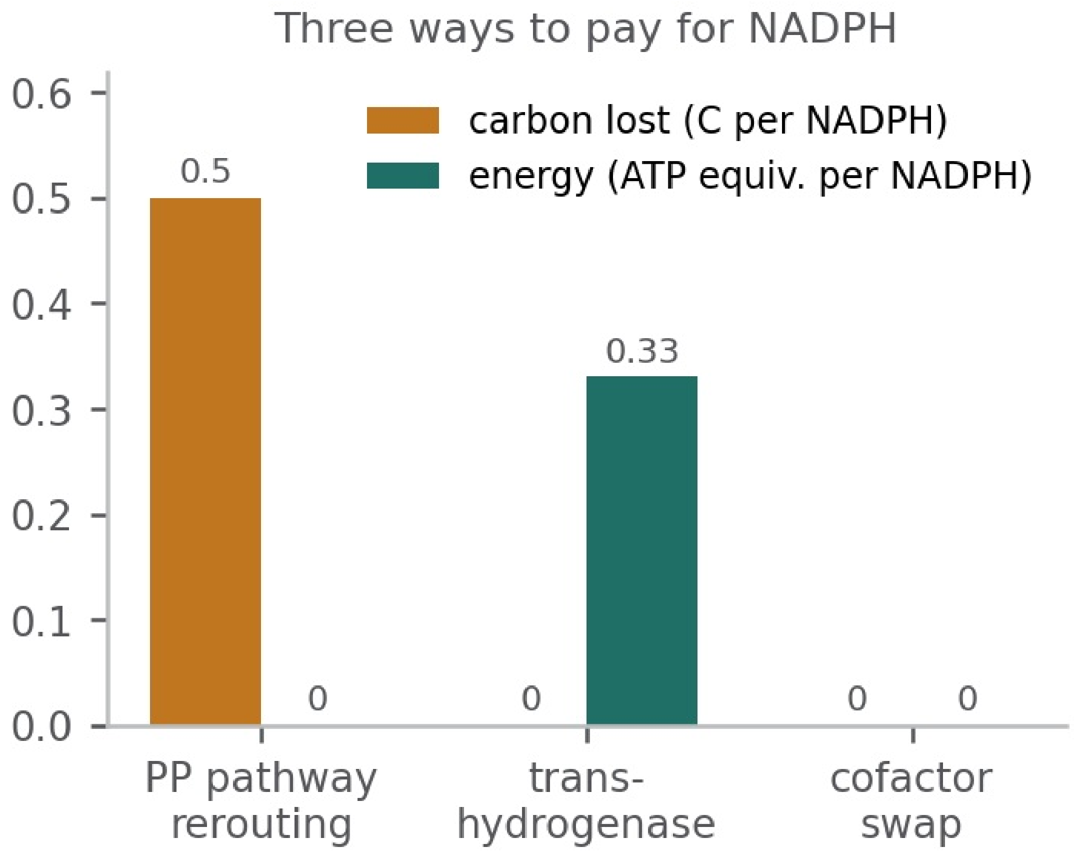


Figure 19.1 The cost of supplying one NADPH by each route. Pentose phosphate rerouting loses one carbon as CO_2 per two NADPH, so 0.5 C per NADPH; a proton-translocating transhydrogenase spends roughly one proton of the motive force per hydride, which at three to four protons per ATP is about 0.3 ATP equivalents; a cofactor swap has no metabolic cost at all, which is why it is attractive and why it is hard. Stoichiometric arithmetic, not measured data.

- **Reroute through the oxidative pentose phosphate pathway.** Each glucose-6-phosphate through the oxidative branch gives 2 NADPH and loses one carbon as CO_2, so half a carbon per NADPH. For a product needing 4 NADPH, that is two carbons lost before you have made anything, and it is why part 1's lysine figure tops out at 0.81 mol/mol rather than 1.0. Easy to implement — deleting phosphoglucose isomerase forces it — and expensive in exactly the currency a commodity process cannot spare.
- **Install or amplify a transhydrogenase.** The membrane-bound, proton-translocating enzyme converts NADH to

NADPH at the cost of the proton motive force, roughly one proton per hydride, so about a third of an ATP equivalent. Cheap in carbon, which is why it is the right first move for a product whose yield is carbon-limited rather than energy-limited. The trap is the soluble transhydrogenase, which runs the other way: express both strongly and you have built a futile cycle that burns ATP equivalents and produces nothing, a mistake that appears in the literature more than once.

- **Swap the enzyme's cofactor specificity.** Change the pathway enzyme so it accepts NADH rather than NADPH. There is no metabolic cost whatsoever, which makes it the best answer in principle; the cost is protein engineering, usually a handful of residues in the binding pocket, and the risk is that the variant has a lower k_{cat} than the parent, so you trade a cofactor problem for a kinetic one. It has been done many times and is now a reasonable first attempt rather than a research project.

Which to choose is a quantitative question and the answer is set by what limits you. Carbon-limited commodity: transhydrogenase or swap. Energy-limited or oxygen-limited process: pentose phosphate, since carbon is cheaper than oxygen transfer. High-value product where neither binds: whichever is fastest to build.

Driving force again, in a design context

Week 14's max-min driving force belongs in this diagnosis, because a cofactor fix can change the thermodynamics. Reductions driven by NADPH have a different driving force from the same reactions driven by NADH, since the two pools are held at very different ratios — NADPH kept reduced, NADH kept oxidised, as part 1 chapter 4 explained. A cofactor swap that solves your supply problem can therefore weaken the driving force of the very step you swapped, and the check is two minutes in eQuilibrator with the appropriate ratio. The same applies in reverse: a step that is marginal with NADH may become comfortable with NADPH, which is occasionally a reason to *increase* your NADPH demand deliberately.

Toxicity, and why dynamic control beats static expression

Two different problems travel under the word toxicity. **Product toxicity** sets the titer ceiling and is mostly a tolerance problem, addressed by in situ removal (weeks 44 to 47), by two-phase fermentation, or by evolving the host — and tolerance is still, honestly, evolved rather than designed, because it is usually polygenic and involves membrane composition and efflux in ways that resist rational design. **Intermediate toxicity** is a pathway-balance problem and is much more tractable, because it arises from a mismatch between the rate of the step that makes the intermediate and the step that consumes it.

That is why dynamic control helps where static overexpression does not. Statically, you choose one expression level for the whole fermentation, and you must choose it low enough that the toxic intermediate never accumulates dangerously — which means it is too low during the phase when the downstream enzyme is abundant. Dynamically, a sensor of the intermediate, or of growth phase, or of cell density, raises the downstream enzyme when the intermediate rises and throttles the upstream one when it rises too far. The pathway then holds the intermediate near a set point instead of at a compromise, which is a control problem of a type you already know how to analyse: a feedback loop with a sensor, an actuator and a set point, and with the same stability questions you would ask of any controller.

Compartmentalisation addresses the same problem differently and should be named alongside it: confining the pathway to an organelle or a protein scaffold raises the local intermediate concentration where the enzymes are while keeping the cytosolic concentration — the one that poisons the cell — low. The costs are transport of every substrate and cofactor across the compartment boundary, and they are quantified in week 58.

Where the surplus electrons actually go, and what that costs in oxygen

A surplus of NADH is usually waved away with the word respiration, but respiration is a transfer to oxygen and oxygen has to be delivered. The stoichiometry is simple: each NADH oxidised by the chain consumes half a mole of O_2, so a pathway with a surplus of two NADH per product imposes one mole of oxygen demand per mole of product on top of whatever growth already needs.

Make it concrete. Producing 2 g/L/h of a 100 Da product is 20 mmol/L/h of product, so a surplus of two NADH apiece is 40 mmol/L/h of NADH and therefore 20 mmol/L/h of extra oxygen demand. Industrial fermenters achieve oxygen transfer rates of the order of 100 to 300 mmol/L/h, depending on vessel, agitation and pressure, so this particular demand is affordable — but it is a tenth to a fifth of the budget before the cells have grown, and a design with a surplus of eight NADH per product at the same rate would not be. The calculation takes a minute and belongs in the same page as the carbon balance, because oxygen transfer is the hard physical ceiling of aerobic fermentation and the thing that degrades most on scale-up.

Two refinements worth carrying. Oxygen transfer costs power, roughly as a strong function of agitation rate, so the last increments are disproportionately expensive, and pushing oxygen demand is a decision about operating cost and not only about feasibility. And the heat follows the oxygen: oxidising substrate releases heat in near-fixed proportion to oxygen consumed, so an oxygen-demanding process is also a cooling-demanding one, which is the real constraint in a large vessel in a warm climate. Both are quantified in phase 4, and both are decided here, in the redox balance.

Tolerance: why it is still evolved rather than designed

The honest state of the art deserves stating plainly, because it is an exception to everything else in this part. Product tolerance is usually polygenic, involving membrane lipid composition, efflux capacity, proton homeostasis, chaperones and general stress responses, and the causal mutations found by evolution are frequently in regulators whose connection to the phenotype is not obvious even in hindsight. Rational design has a poor record here, and adaptive laboratory evolution — serial passage under increasing product concentration, followed by sequencing the survivors — has a good one.

Treat that as a method rather than a defeat, and run it properly. Evolve in the condition you actually care about, including the product at the concentration you need and the medium you will use, because evolution optimises what you select for and nothing else. Keep the populations replicate and independent, so that convergent mutations across lineages identify real targets rather than one lineage's history. Sequence and then reconstruct the candidate mutations individually in the clean parent, because an evolved strain carries passengers as well as drivers, and only the reconstruction tells you which is which. Done that way, evolution becomes a hypothesis generator feeding the design stage of chapter 11's loop, and the mutations it finds are reusable knowledge.

Order of balances	carbon → NADH → NADPH → ATP, separately, per mole of product
PP pathway cost	2 NADPH per glucose-6-P, 1 CO_2 lost → 0.5 C per NADPH
Transhydrogenase cost	≈1 H^+ per hydride ≈ 0.3 ATP equivalents per NADPH
Cofactor swap cost	zero metabolic cost; risk is k_cat of the engineered variant
Dynamic control	hold the intermediate at a set point instead of at a compromise

Your pathway consumes 4 NADPH and 2 ATP per product and yields 4 of glucose's 6 carbons. Diagnose it and propose a route to the highest yield you can defend.

Carbon first: 4 of 6 carbons reach the product, so the naive ceiling is 0.67 mol/mol on carbon alone. Redox: 4 NADPH are needed and glycolysis supplies NADH, not NADPH. If you pay for all of it through the oxidative pentose phosphate pathway, 4 NADPH needs 2 glucose-6-phosphate through the branch and costs 2 carbons as CO_2 — which, with the carbon skeletons returning to glycolysis, drops the achievable yield well below the carbon ceiling, which is exactly the lysine result from part 1. ATP: glycolysis nets 2 and the pathway spends 2, so there is nothing for growth before maintenance, which forces a two-stage process or a partial respiration of some carbon.

The defensible route: swap as much of the NADPH demand to NADH as the enzymes allow, since glycolysis already makes NADH and the swap costs no carbon; cover the residual demand with a proton-translocating transhydrogenase, paying ~0.3 ATP per NADPH rather than 0.5 carbon; and accept that the ATP balance now requires a growth phase that is decoupled from production. Then state the assumption that will decide whether this works — that the engineered enzymes retain enough activity — and design the experiment that tests it before building the rest.

Three things people get wrong

- ‘Reducing equivalents balance.’ NADH and NADPH balance separately, and lumping them hides the problem that will dominate the project.
- ‘Add a transhydrogenase and the redox problem goes away.’ It moves it into the energy budget, and expressing both directions builds a futile cycle.
- ‘Toxicity is a tolerance problem.’ Product toxicity is; intermediate toxicity is a pathway balance problem and is fixable by control.

Self-check — answer on paper first

1. Give the cost, in its own units, of each of the three standard NADPH fixes, and name the process condition that decides between them.

Pentose phosphate rerouting costs half a carbon per NADPH, lost as CO_2; a proton-translocating transhydrogenase costs roughly a third of an ATP equivalent per NADPH, taken from the proton motive force; a cofactor swap costs nothing metabolically and risks the enzyme's turnover number. What decides is which resource binds: carbon for a commodity whose economics are feedstock-dominated, energy or oxygen transfer for a process already at its respiratory limit.

2. Why does a toxic intermediate argue for dynamic rather than static control, and what does the controller need?

Because static expression forces one compromise level for the whole run: low enough to keep the intermediate safe at the worst moment, and therefore too low whenever conditions are better, so the pathway is throttled most of the time. A dynamic controller holds the intermediate near a set point by raising consumption or lowering production as it rises. It needs three things: a sensor that responds to the intermediate or to a correlate of it, an actuator — a promoter, small RNA or degradation tag that changes an enzyme's level fast enough — and a loop whose gain and delay do not make it oscillate, which is the stability question you would ask of any controller.

3. A design balances carbon and total reducing equivalents but needs 3 NADPH and produces 3 NADH per product. What is wrong and what are the options?

Nothing is wrong on paper and everything is wrong in the cell: the two pools are held at deliberately different ratios and are not interchangeable, so the design has a 3 NADPH deficit and a 3 NADH surplus simultaneously. Options: a transhydrogenase, which solves both at once and is the natural answer here, at roughly an ATP equivalent in total; swapping pathway enzymes to NADH, which solves both at no metabolic cost if the enzymes allow it; or oxidising the NADH surplus through respiration while paying for NADPH through the pentose phosphate pathway, which solves both while losing 1.5 carbons and is the worst of the three on yield.

What to open a book for. Hartline et al., Metabolic Engineering 63:126 (2021), for dynamic control — it is the review to know by name. Re-read Nielsen & Keasling (2016) with cofactors in mind; the second reading is a different paper. For the balances themselves, part 1 chapter 4 and Stephanopoulos Ch. 2 and 3 are all you need.

The phase 2 checkpoint

Answer all four from memory, on paper, before reading the model answers. If the first one is shaky, the problem is the balances in part 1 chapter 4 and you should go back to them rather than forward.

1. For a target of your choice, write the maximum-yield stoichiometry from glucose and identify the cofactor surplus or deficit.

Method, not answer. Write the balanced synthesis from glucose; close the carbon balance; close NADH and NADPH separately; close ATP. A surplus of NADH means the pathway is more oxidised than the substrate and needs an electron sink or respiration. A deficit of NADPH means pentose phosphate flux or a transhydrogenase must supply it, at the carbon or energy cost you should state in the answer. An ATP deficit means nothing is left for growth, which forces a staged process. A good answer states the number and its cost; an excellent one states which of the four balances binds.

2. State the MCA summation theorem and its engineering implication.

The flux control coefficients of all enzymes in a pathway sum to 1. Control is therefore shared: no single enzyme has a coefficient of 1, so raising one enzyme gives a sublinear flux gain and the limitation simply moves to another step — in the model of chapter 15, tenfold overexpression of a step with a coefficient of 0.10 bought 8% more flux and left that step with no control at all. Useful improvements require several coordinated changes, which is the argument for multiplexed editing and for library-based tuning.

3. Define titer, rate and yield, and say which dominates for a bulk chemical and which for a high-value pharmaceutical.

Titer is concentration at harvest (g/L); rate is volumetric productivity (g/L/h), quoted over the whole fermentation; yield is product per substrate (g/g or mol/mol), best reported as a fraction of theoretical. For a bulk chemical, yield binds because feedstock dominates cost — $0.40/kg glucose at 0.25 g/g is $1.60 of sugar per kilogram of product — and titer matters next because a dilute broth makes separation expensive. For a high-value pharmaceutical, total output and purity dominate and a few per cent of sugar is irrelevant.

4. Read one Metabolic Engineering abstract, predict the design logic, then check yourself against the paper.

No fixed answer. You are checking whether, from the abstract alone, you could name the host, the likely bottleneck and the type of intervention before reading the paper. If you could not, reread weeks 11 to 19 rather than pressing on.

Before you go on to phase 3

Phase 3 is the technique phase: assembly, editing, screening, analytics. It is much more concrete than what you have just done, and the risk is the opposite one — that you will enjoy the protocols and stop asking what they are for. The discipline to carry forward is that every technique in phase 3 exists to widen one stage of the loop in chapter 11, and the question to ask of each is which stage, and by how much.

Three things from this phase you will use on nearly every page of what follows: that stoichiometry constrains and measurements decide, so every flux number you read was partly assumed; that control is distributed, so every single-gene result you read should be checked against the fold-change it required; and that the honest reporting of an experiment includes what it could have detected. If you keep those three, you will read the literature more usefully than most of the people writing it.

On the numbers in this part

Everything quantitative here was computed by a script rather than remembered, and the scripts are short enough to retype. The three-enzyme model in chapter 16 generated figures 15.1 and 16.1 and every control coefficient quoted in chapter 15; its parameters are assumed, and the captions say so. The stiffness comparison in chapter 12 was measured on the Robertson problem at stated tolerances — 243,050 right-hand-side evaluations against 305 — and will vary a little with SciPy version. The statistics in chapter 17 come from the tracker's own example data, computed with scipy.stats, with power from the non-central t distribution rather than a normal approximation. The thermodynamic and transport numbers in chapter 14 use the assumptions stated beside them, and the permeability coefficients are order-of-magnitude class values rather than measurements for any particular molecule.

Where I have given a figure that matters to a decision, check it: eQuilibrator for free energies, BioNumbers for physiological concentrations and cell parameters, and the primary paper for anything attributed to one. The errors in this document are mine. What it leaves out deliberately is the full derivation of the control-analysis theorems, the detailed treatment of labelling-based flux analysis, which belongs with the analytics in phase 3, and genome-scale modelling practice, which needs the software in front of you rather than a page.